

```
# IMPORTANT: SOME KAGGLE DATA SOURCES ARE PRIVATE
# RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES.
import kagglehub
kagglehub.login()
```

```
# IMPORTANT: RUN THIS CELL IN ORDER TO IMPORT YOUR KAGGLE DATA SOURCES,
# THEN FEEL FREE TO DELETE THIS CELL.
# NOTE: THIS NOTEBOOK ENVIRONMENT DIFFERS FROM KAGGLE'S PYTHON
# ENVIRONMENT SO THERE MAY BE MISSING LIBRARIES USED BY YOUR
# NOTEBOOK.
```

```
jessicali9530_lfw_dataset_path = kagglehub.dataset_download('jessicali9530/lfw-dataset')
averkij_lfw_attributes_path = kagglehub.dataset_download('averkij/lfw-attributes')
fiktus_taskone_path = kagglehub.dataset_download('fiktus/taskone')

print('Data source import complete.')
```



✓ Домашнее задание "Варианционные автоэнкодеры"

В этом домашнем задании вам предстоит реализовать несколько версий автоэнкодера для работы с изображениями.

Вы научитесь добавлять улыбку на фотографии людей, изменяя их исходное выражение лица с помощью обученной модели. А также, обучите ваш автоэнкодер генерировать изображения рукописных цифр.

✓ Часть 1. Vanilla Autoencoder (10 баллов)

✓ 1.1. Подготовка данных (1 балл)

Перед тем, как начинать работу с моделями нужно подготовить данные.

```
import numpy as np
from torch.autograd import Variable
from torchvision import datasets
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data_utils
import torch
import matplotlib.pyplot as plt

import os
import pandas as pd
import skimage.io
from skimage.transform import resize

import imageio
from PIL import Image
import kagglehub

device = 'cuda' if torch.cuda.is_available() else 'cpu'

%matplotlib inline
```

В этом задании вам предстоит работать с датасетом [LFW \(Labeled Faces in the Wild\)](#).

```
# Скачиваем картинки
images_path = kagglehub.dataset_download("jessicali9530/lfw-dataset")
print("Path to dataset files:", images_path)
```

```
Path to dataset files: /kaggle/input/datasets/jessicali9530/lfw-dataset
```

```
# Скачиваем атрибуты
attrs_path = kagglehub.dataset_download("averkij/lfw-attributes")
print("Path to dataset files:", attrs_path)
```

```
Path to dataset files: /kaggle/input/datasets/averkij/lfw-attributes
```

```
#DATASET_PATH = "/kaggle/input/lfw-dataset/lfw-deepfunneled/lfw-deepfunneled/"
#ATTRIBUTES_PATH = "/kaggle/input/lfw-attributes/lfw_attributes.txt"
```

```
DATASET_PATH = "/kaggle/input/datasets/jessicali9530/lfw-dataset/lfw-deepfunneled"
ATTRIBUTES_PATH = "/kaggle/input/datasets/averkij/lfw-attributes/lfw_attributes.txt"
```

```

def fetch_dataset(dx=80, dy=80, dimx=45, dimy=45):

    df_attrs = pd.read_csv(ATTRIBUTES_PATH, sep='\t', skiprows=1,)
    df_attrs = pd.DataFrame(df_attrs.iloc[:, :-1].values, columns = df_attrs.columns[1:]

    photo_ids = []
    for dirpath, dirnames, filenames in os.walk(DATASET_PATH):
        for fname in filenames:
            if fname.endswith(".jpg"):
                fpath = os.path.join(dirpath, fname)
                photo_id = fname[:-4].replace('_', ' ').split()
                person_id = ' '.join(photo_id[:-1])
                photo_number = int(photo_id[-1])
                photo_ids.append({'person': person_id, 'imagenum': photo_number, 'photo

    photo_ids = pd.DataFrame(photo_ids)
    df = pd.merge(df_attrs, photo_ids, on=('person', 'imagenum'))

    assert len(df) == len(df_attrs), "Потеряны данные при объединении датафреймов!"

    images = df['photo_path'].apply(imageio.imread)\
        .apply(lambda img: img[dh:-dh, dx:-dx])\
        .apply(lambda img: np.array(Image.fromarray(img).resize(

    images = np.stack(images.values).astype('uint8')
    attrs = df.drop(["photo_path", "person", "imagenum"], axis=1)

    return images, attrs

```

Обратите внимание, что датасет представляет собой не только картинки, но и атрибуты
Атрибуты понадобятся в конце этого задания

```
images, attrs = fetch_dataset()
```

```
images.shape
```

```
(13143, 45, 45, 3)
```

```
attrs
```

	Male	Asian	White	Black	Baby	Child	Youth	Middle Aged
0	1.568346	-1.889043	1.737203	-0.929729	-1.471799	-0.19558	-0.835609	-0.351468
1	0.169851	-0.982408	0.422709	-1.282184	-1.36006	-0.867002	-0.452293	-0.197521
2	0.997749	-1.364195	-0.157377	-0.756447	-1.891825	-0.871526	-0.862893	0.031445
3	1.122719	-1.997799	1.916144	-2.514214	-2.580071	-1.404239	0.057551	0.000196
4	1.078214	-2.008098	1.676211	-2.278056	-2.651845	-1.348408	0.649089	0.017656
...	...	...	...	...	...	...	...	...
13138	-0.205363	-0.202842	-1.232735	-1.409125	-1.804858	0.287268	-0.521815	-0.789165
13139	1.957472	-2.47247	-1.307994	-0.671636	-1.960125	-0.838803	-1.365955	0.510448
13140	-0.037001	-1.16415	0.228494	0.187503	-1.836475	-0.958412	-0.669697	-0.48033
13141	0.282219	-0.396198	1.27047	-1.981209	-1.70294	-1.507203	-2.330721	-0.295928
13142	0.071197	-1.92884	2.122345	-1.892862	-2.044238	-0.313663	-0.081369	-0.140593

Разбейте датасет изображений и атрибутов на train и val, выведите несколько картинок в output, чтобы посмотреть, как они выглядят, и приведите картинки к тензорам pytorch, чтобы можно было скормить их сети:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(images, attrs, test_size = 0.05, s
```

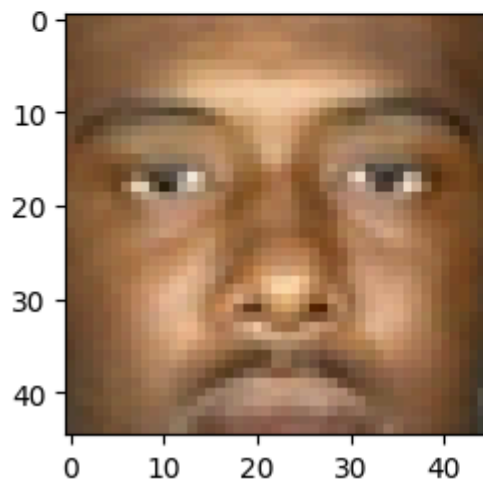
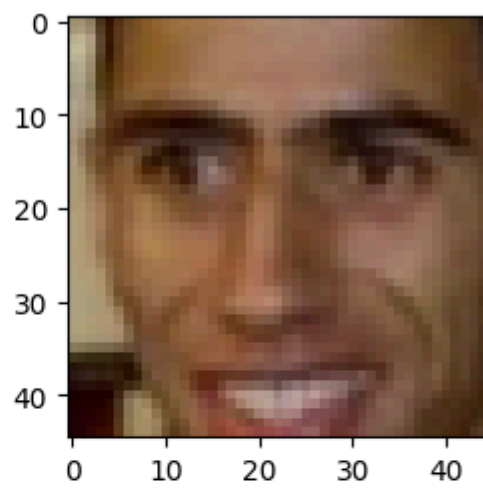
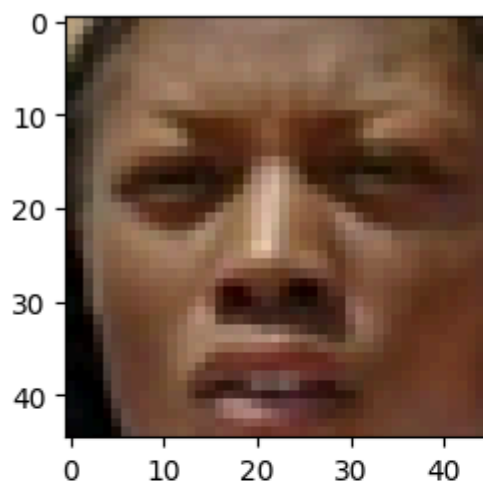
```
print(X_train.shape)
print(y_train.shape)
```

```
(12485, 45, 45, 3)
(12485, 73)
```

```
X_train_tensor = torch.tensor(X_train.transpose(0, 3, 1, 2), dtype=torch.float32) / 255
X_test_tensor   = torch.tensor(X_test.transpose(0, 3, 1, 2), dtype=torch.float32) / 255
```

```
from torch.utils.data import TensorDataset, DataLoader
batch_size = 32
train_dataset = TensorDataset(X_train_tensor)
train_dataloader = DataLoader(train_dataset, batch_size = batch_size, shuffle = True )
test_dataset = TensorDataset(X_test_tensor)
test_dataloader = DataLoader(test_dataset, batch_size = batch_size, shuffle = True )
```

```
import matplotlib.pyplot as plt
_, axes = plt.subplots(2, 2, figsize=(6, 6))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_train[i])
```

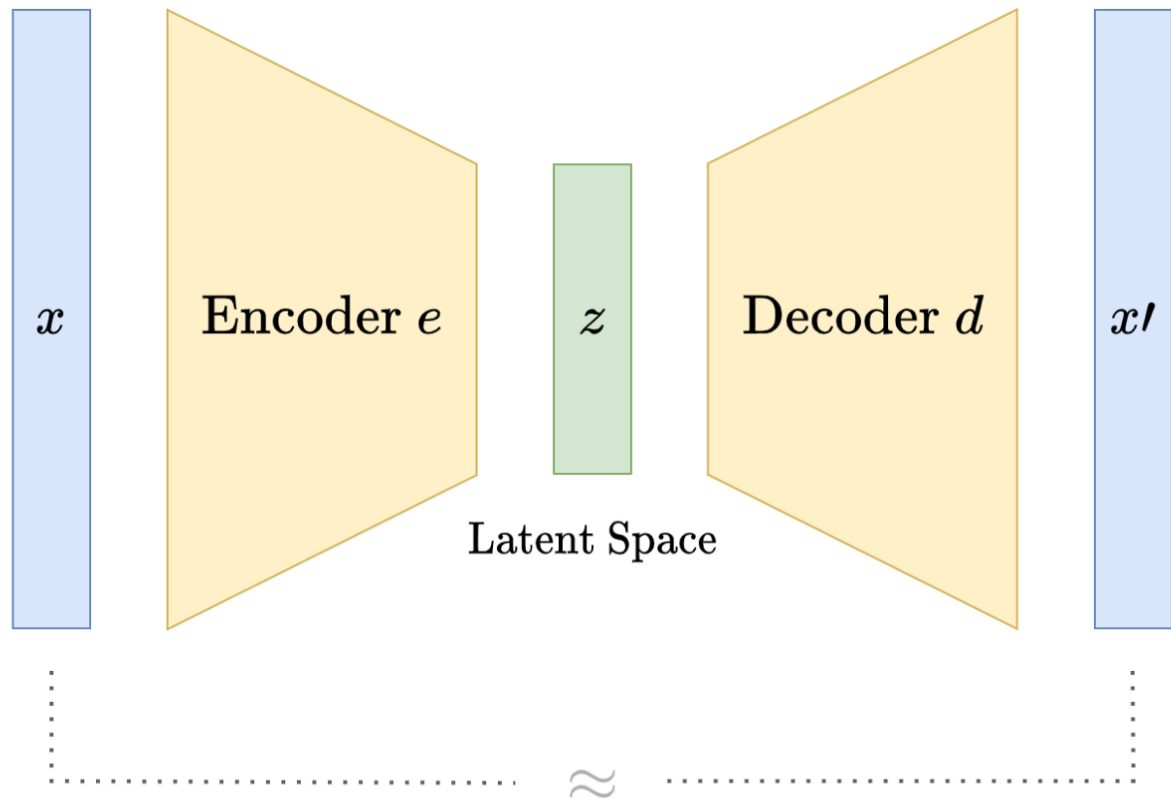


Напишите программный код или сгенерируйте его с помощью искусственного интеллекта.

✓ 1.2. Архитектура модели (2 балла)

В этом разделе вы напишите и обучите обычный автоэнкодер.

General Autoencoder Architecture



```
dim_code = 128 # выберите размер латентного вектора
#дальше я явно указывал 128
```

Реализуем autoencoder. Архитектуру (conv, fully-connected, ReLU, etc) можете выбирать сами. Экспериментируйте!

```
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        self.enc = nn.Sequential(
            nn.Conv2d(3, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten(),
        )

        self.dec = nn.Sequential(
            nn.Unflatten(1, (128, 3, 3)),
            nn.ConvTranspose2d(128, 128, 3, stride=2, padding=1, output_padding=1),
```

```

        nn.ReLU(),
        nn.ConvTranspose2d(128, 64, 3, stride=2, padding=1, output_padding=1),
        nn.ReLU(),
        nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1),
        nn.ReLU(),
        nn.ConvTranspose2d(32, 16, 3, stride=2, padding=1),
        nn.ReLU(),
        nn.ConvTranspose2d(16, 3, 3, stride=1, padding=1),
        nn.ReLU(),

    )

    def forward(self, x):
        latent_code = self.enc(x)
        reconstruction = self.dec(latent_code)
        # реализуйте forward проход автоэнкодера
        # в качестве возвращаемых переменных -- латентное представление картинки (latent_code)
        # и полученная реконструкция изображения (reconstruction)

        return reconstruction, latent_code

```

```

import torch.optim as optim

criterion = nn.MSELoss()

model = Autoencoder().to(device)

optimizer = optim.Adam(model.parameters(), lr = 3e-4 )

```

✓ 1.3 Обучение (2 балла)

Осталось написать код обучения автоэнкодера. При этом было бы неплохо в процессе иногда смотреть, как автоэнкодер реконструирует изображения на данном этапе обучения. Например, после каждой эпохи (прогона train выборки через автоэнкодер) можно смотреть, какие реконструкции получились для каких-то изображений val выборки.

А, ну еще было бы неплохо выводить графики train и val лоссов в процессе тренировки =)

```

from tqdm import tqdm
epoch = 50
train_loss = []
test_loss = []
val_vis = X_test_tensor[0:1].to(device)
for i in tqdm(range(epoch)):
    model.train()
    epoch_loss = {'train': [], 'test': []}

    for batch in train_dataloader:
        optimizer.zero_grad()
        batch = batch[0].to(device)

```

```

    pred = model(batch)

    loss = criterion(pred[0], batch)

    loss.backward()
    optimizer.step()
    epoch_loss['train'].append(loss.item())
    eplosstr = np.mean(epoch_loss['train'])
    train_loss.append(eplosstr)

model.eval()
for batch in test_dataloader:
    with torch.no_grad():
        batch = batch[0].to(device)
        pred = model(batch)
        loss = criterion(pred[0], batch)
        epoch_loss['test'].append(loss.item())
    eplosstest = np.mean(epoch_loss['test'])
    test_loss.append(eplosstest)

with torch.no_grad():
    recon_vis, _ = model(val_vis)

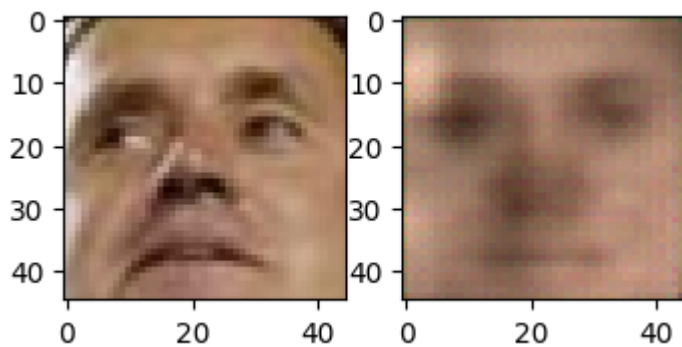
fig, axes = plt.subplots(1, 2, figsize=(4,2))
axes[0].imshow(val_vis[0].cpu().permute(1,2,0).clamp(0,1))
axes[1].imshow(recon_vis[0].cpu().permute(1,2,0).clamp(0,1))
plt.show()

plt.figure(figsize=(6,3))
plt.plot(train_loss, label='train')
plt.plot(test_loss, label='val')

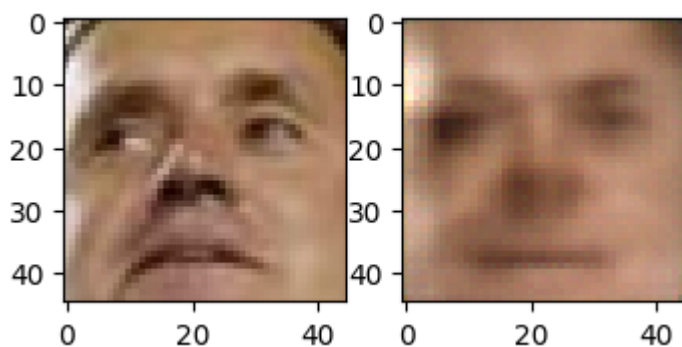
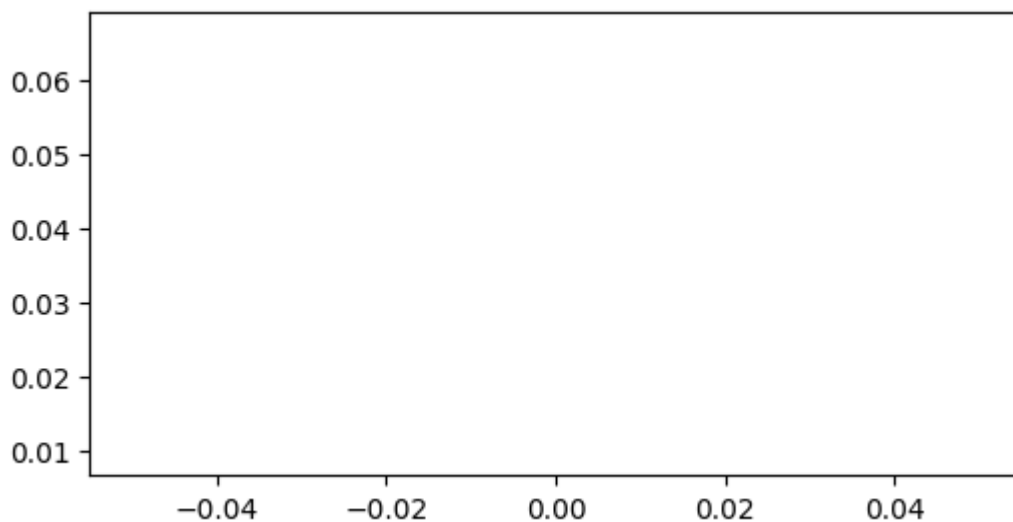
print(f'epoch:{i}, cur train loss: {eplosstr}, curr test loss: {eplosstest}')

```

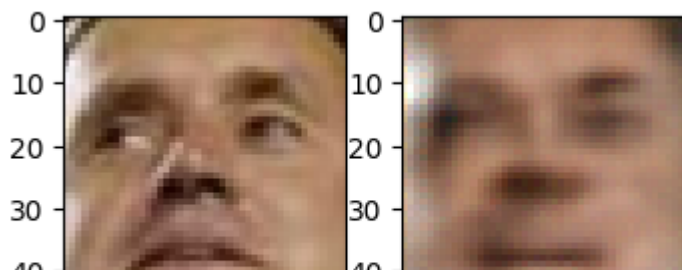
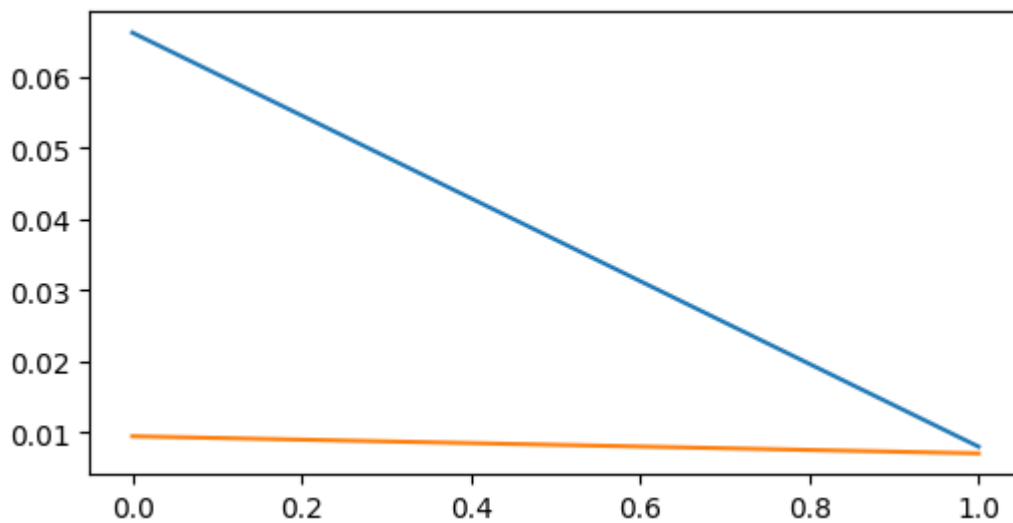

0%| | 0/50 [00:00<?, ?it/s]



2%|| | 1/50 [00:03<03:00, 3.68s/it]epoch:0, cur train loss: 0.0662022507523

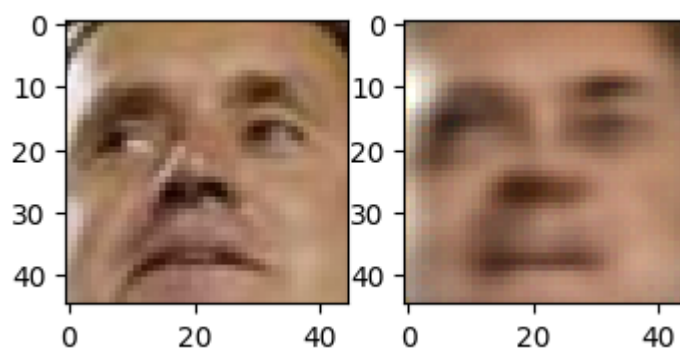
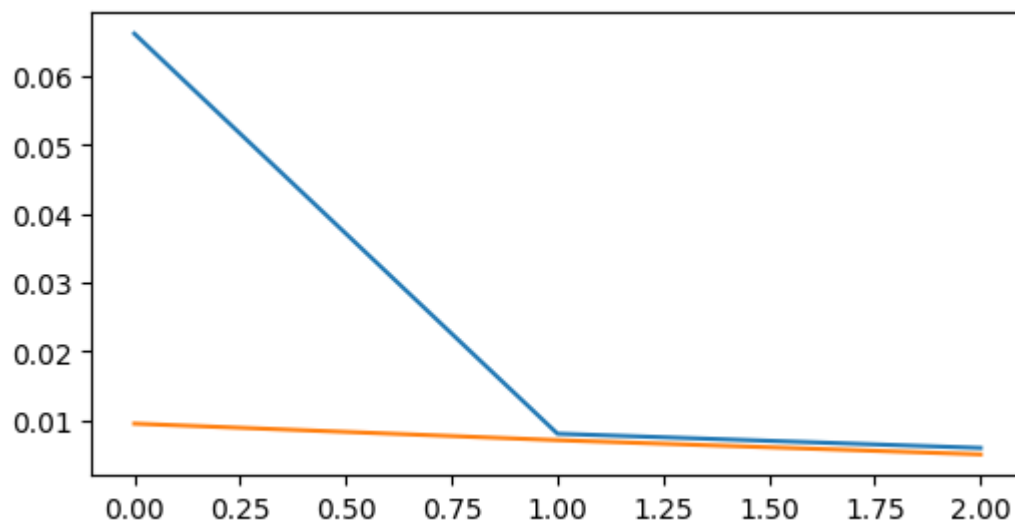


4%|| | 2/50 [00:05<01:57, 2.44s/it]epoch:1, cur train loss: 0.0080455528986

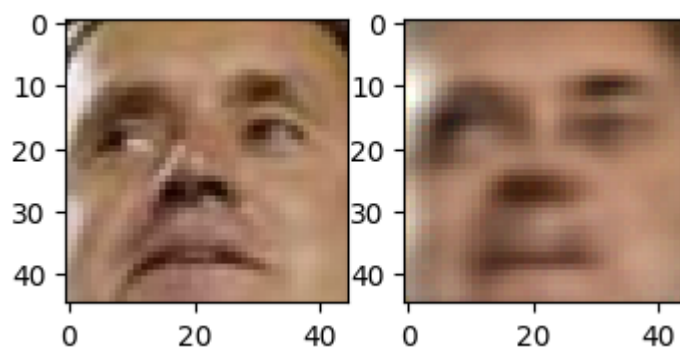
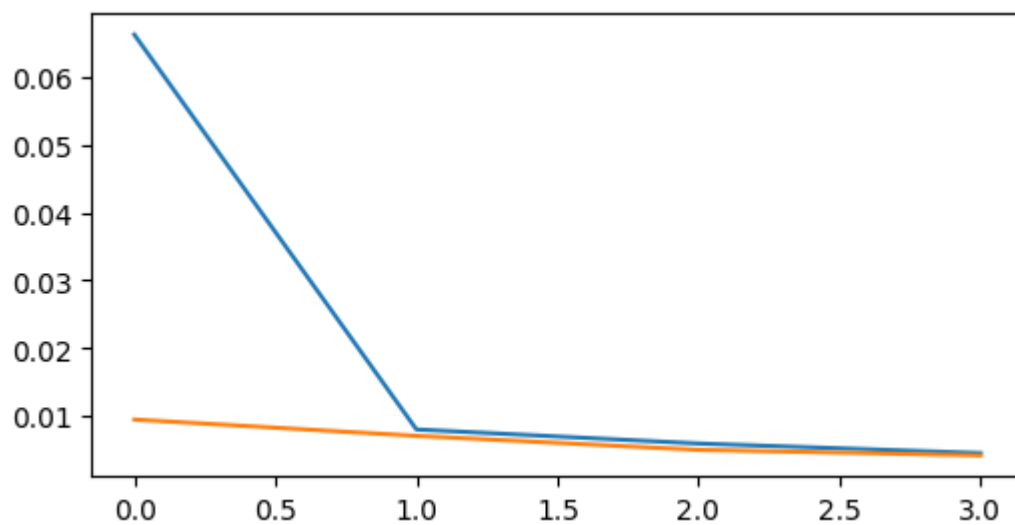




6% | 3/50 [00:06<01:36, 2.06s/it]epoch:2, cur train loss: 0.00597502029193

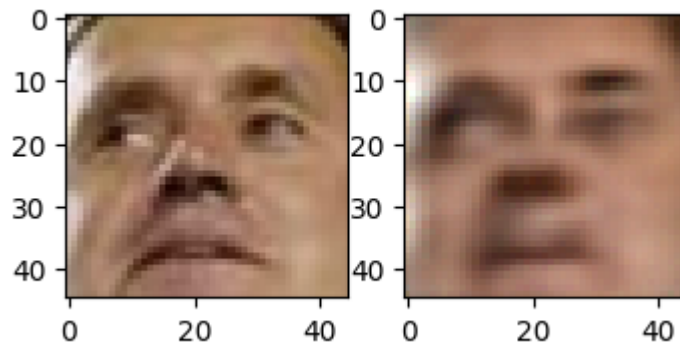
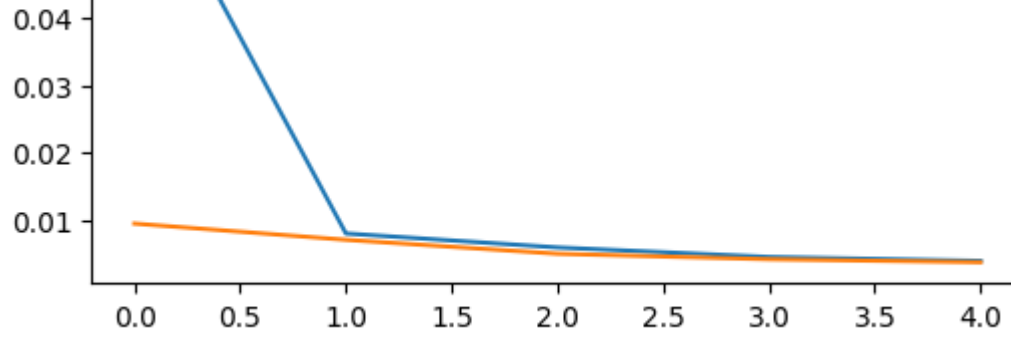


8% | 4/50 [00:08<01:26, 1.87s/it]epoch:3, cur train loss: 0.0045611091819

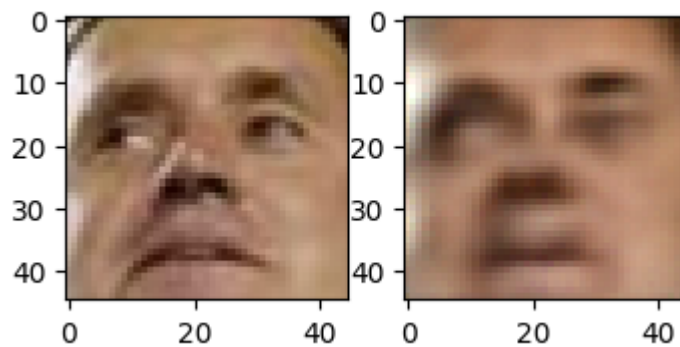
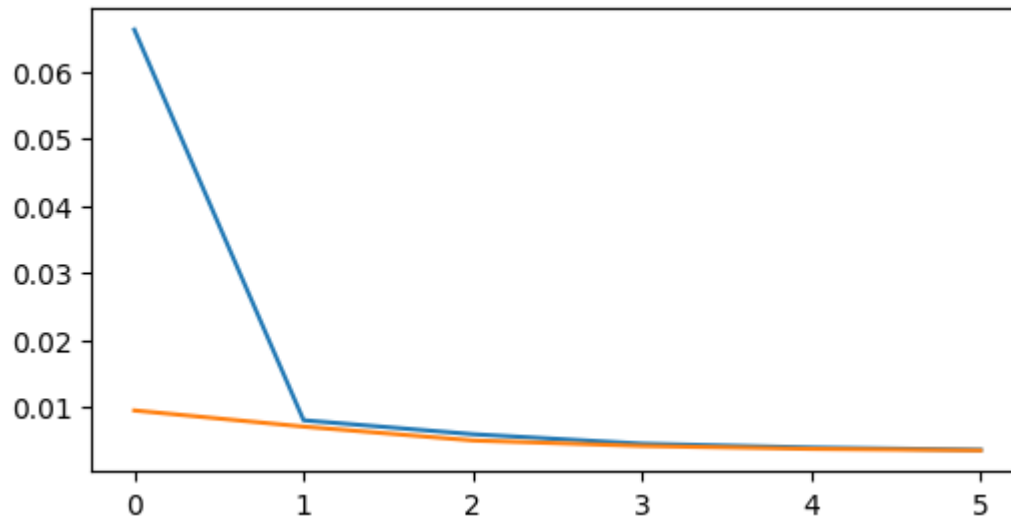


10% | 5/50 [00:10<01:19, 1.77s/it]epoch:4, cur train loss: 0.00398971970774

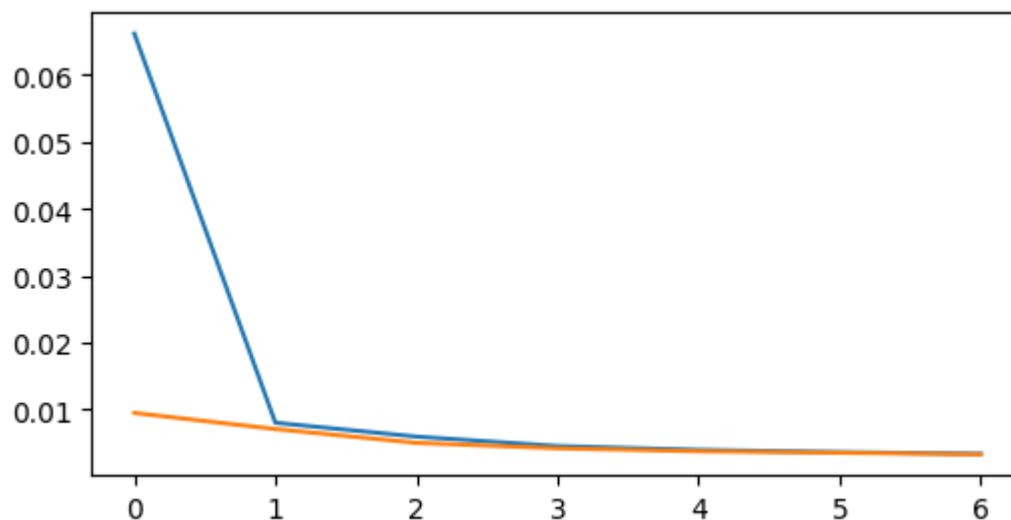


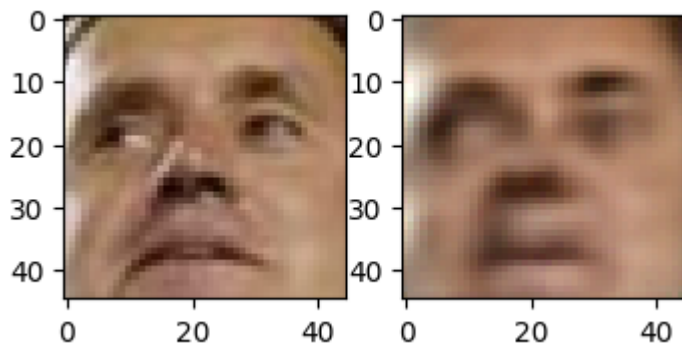


12% | 6/50 [00:11<01:15, 1.72s/it]epoch:5, cur train loss: 0.0036510298328

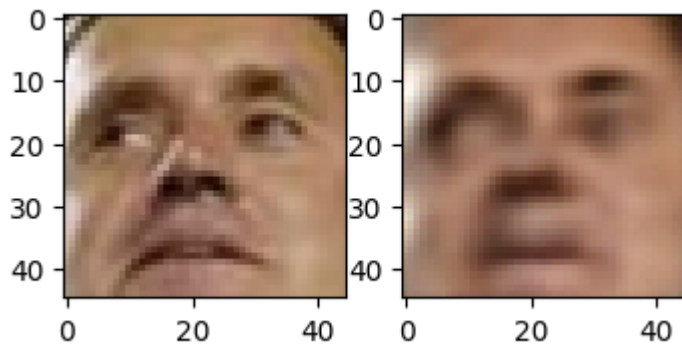
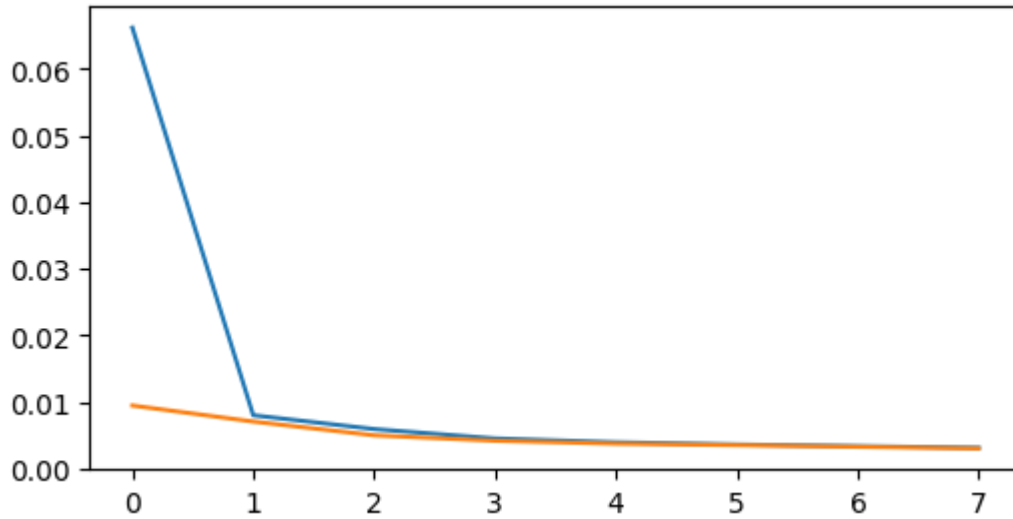


14% | 7/50 [00:13<01:12, 1.70s/it]epoch:6, cur train loss: 0.0034048293498

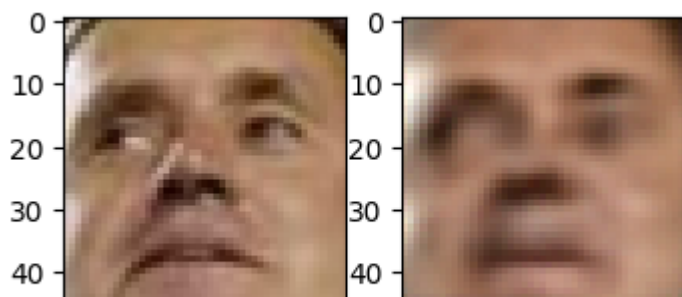
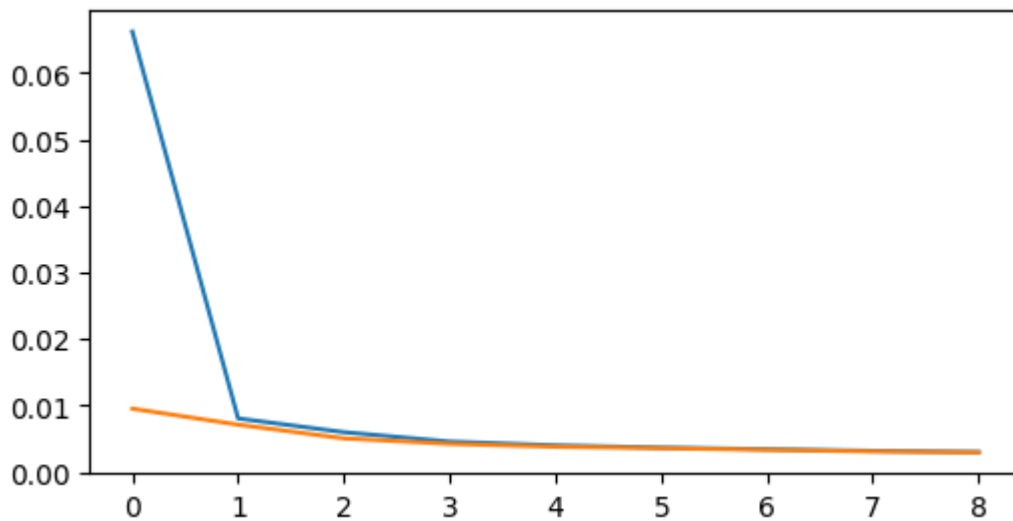




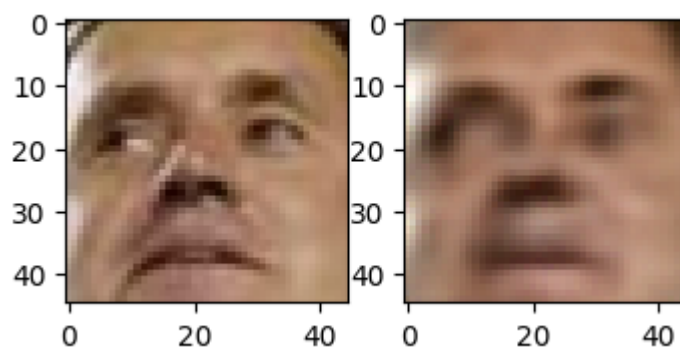
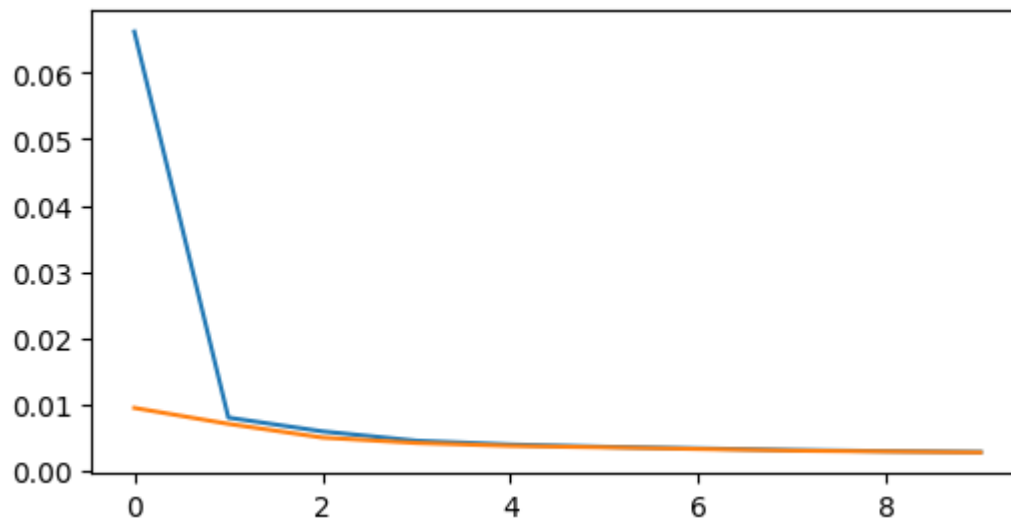
16% |███ | 8/50 [00:14<01:09, 1.66s/it]epoch:7, cur train loss: 0.00316348509229



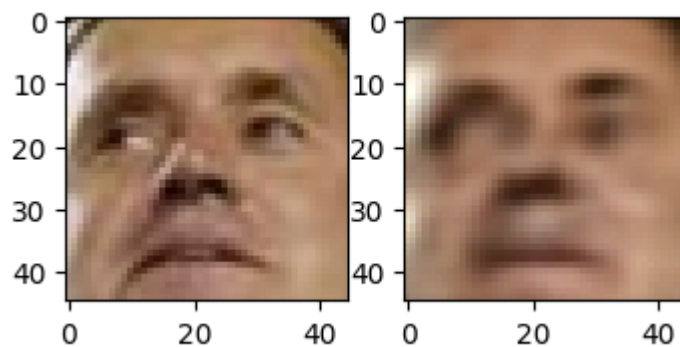
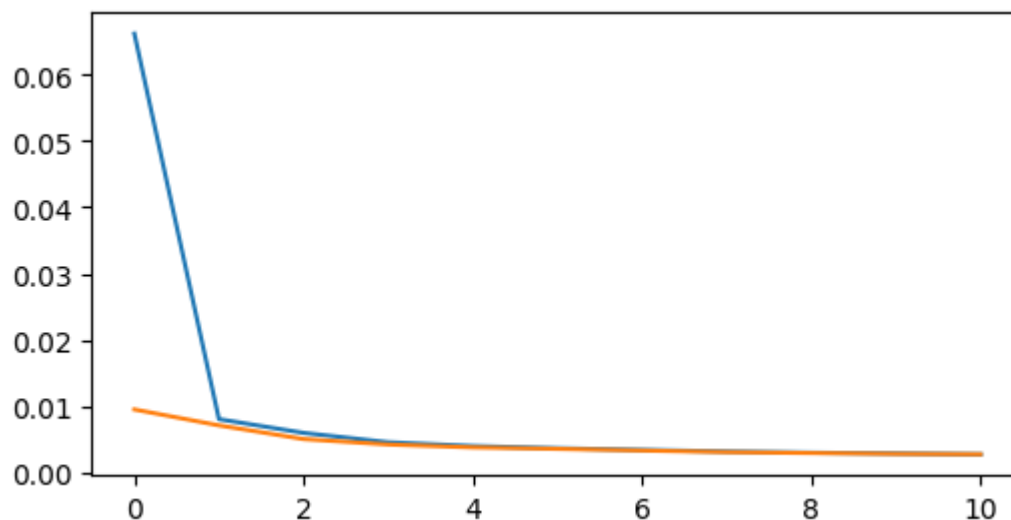
18% |███ | 9/50 [00:16<01:07, 1.65s/it]epoch:8, cur train loss: 0.0029882805277



0 20 40 0 20 40
20% | 10/50 [00:18<01:05, 1.63s/it]epoch:9, cur train loss: 0.0028663969984

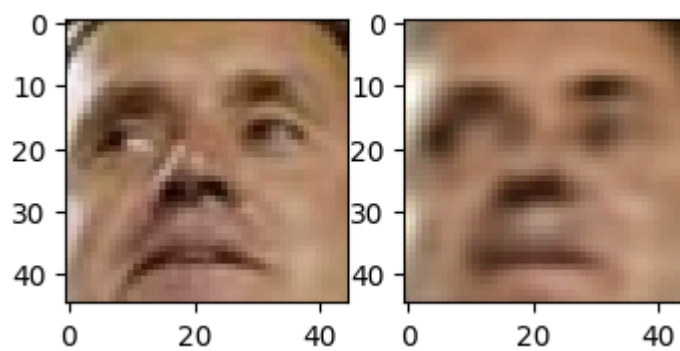
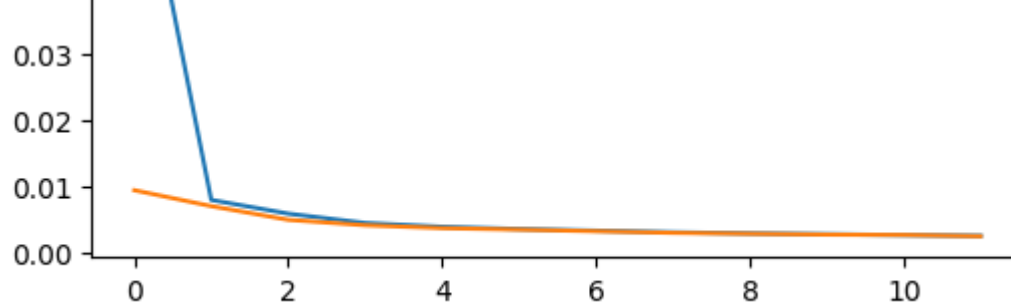


22% | 11/50 [00:19<01:03, 1.62s/it]epoch:10, cur train loss: 0.00273365396

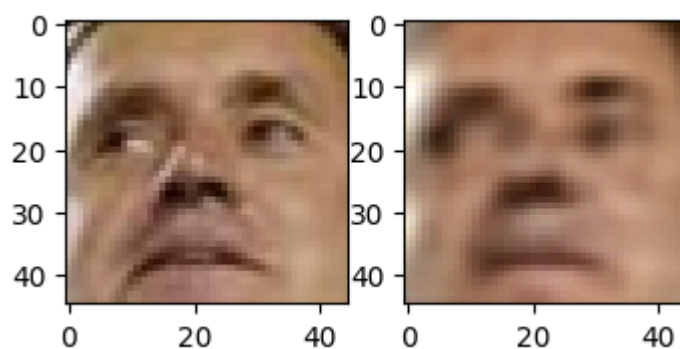
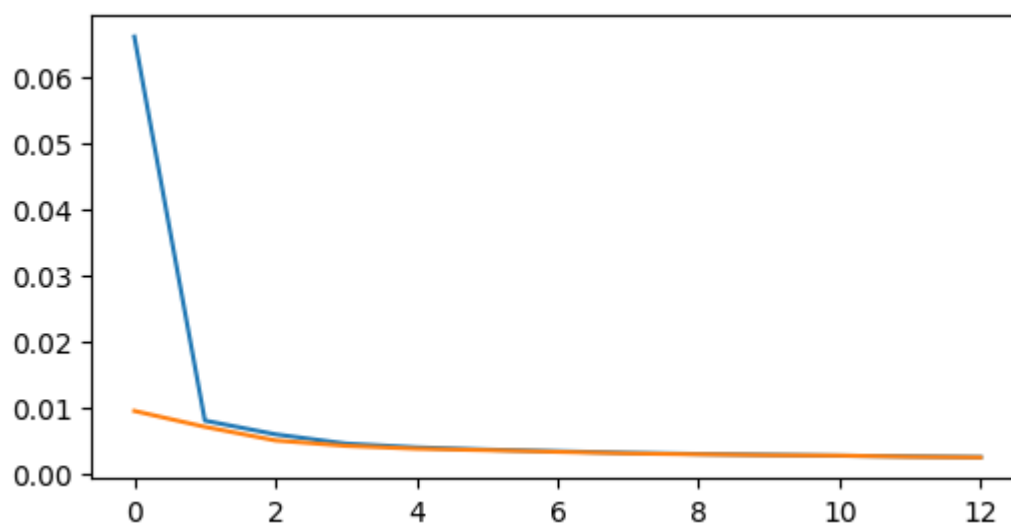


24% | 12/50 [00:21<01:01, 1.62s/it]epoch:11, cur train loss: 0.00262506034

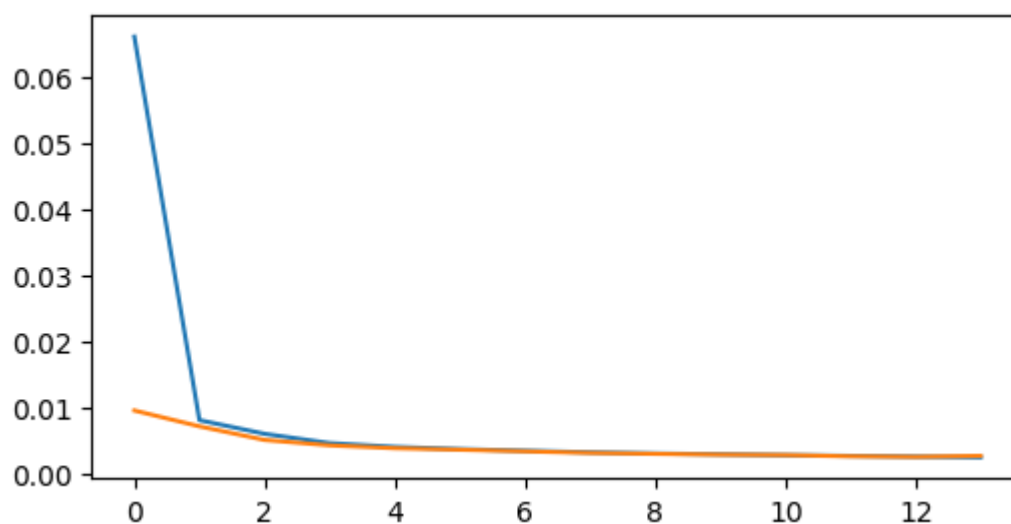


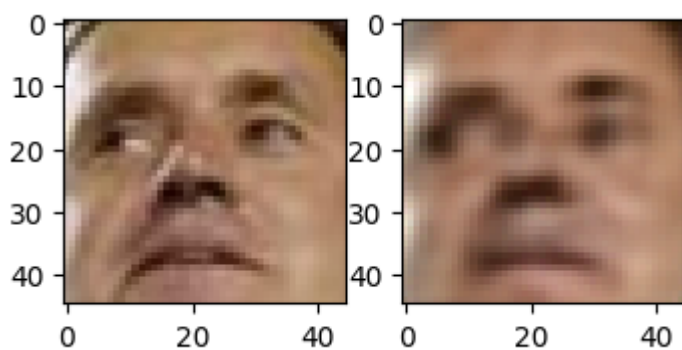


26% |■■■■■ | 13/50 [00:23<01:02, 1.70s/it]epoch:12, cur train loss: 0.002513376796

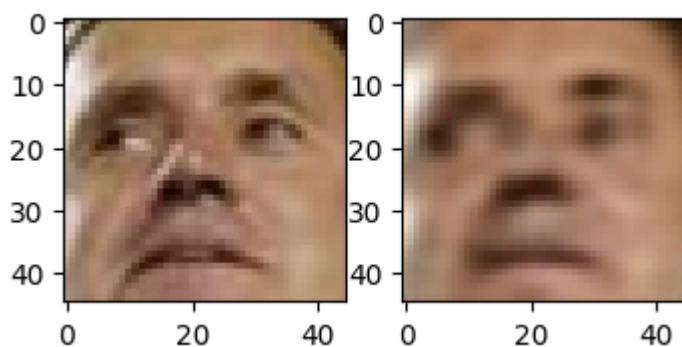
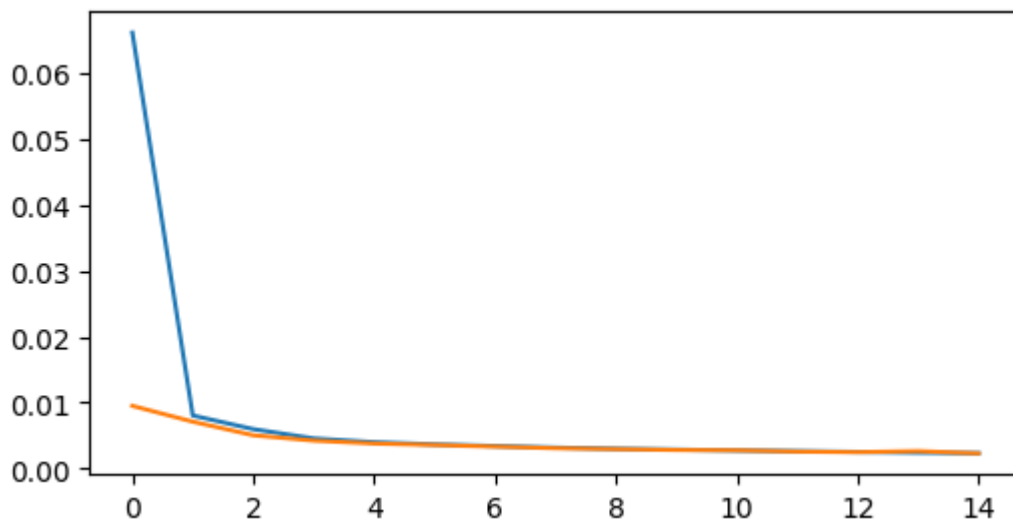


28% |■■■■■ | 14/50 [00:24<01:00, 1.67s/it]epoch:13, cur train loss: 0.00240087180

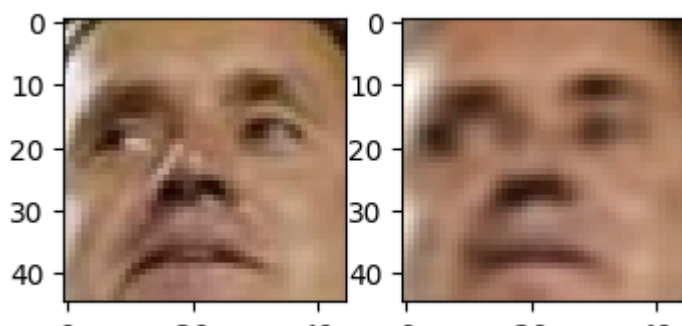
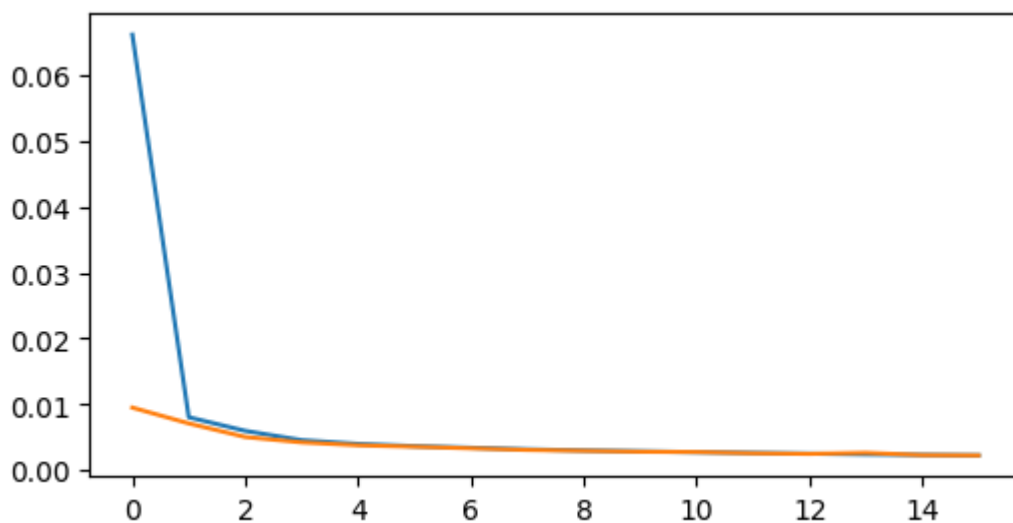




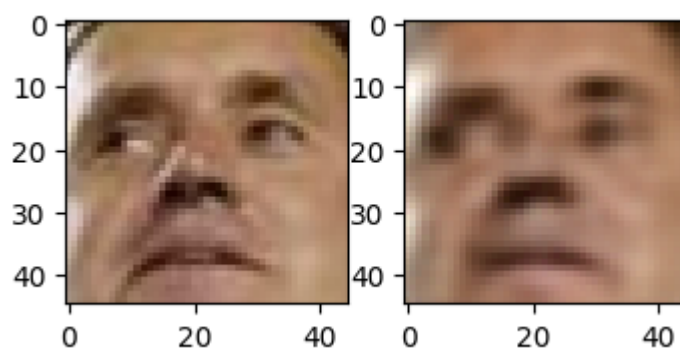
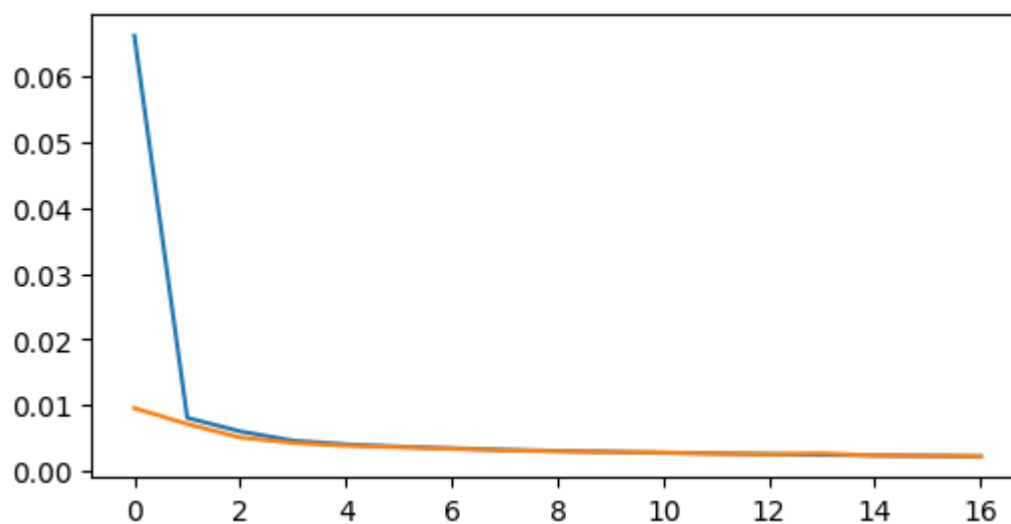
30%|███████| 15/50 [00:26<00:57, 1.64s/it]epoch:14, cur train loss: 0.002322944281



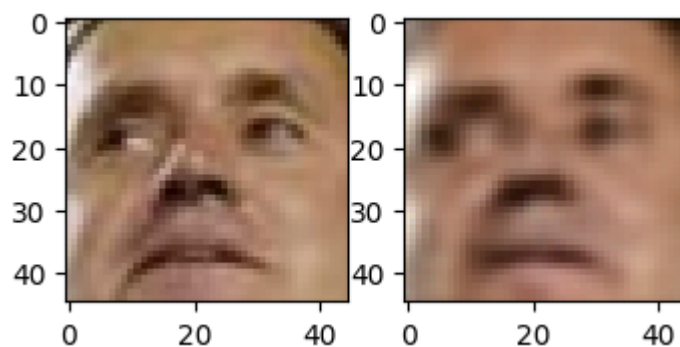
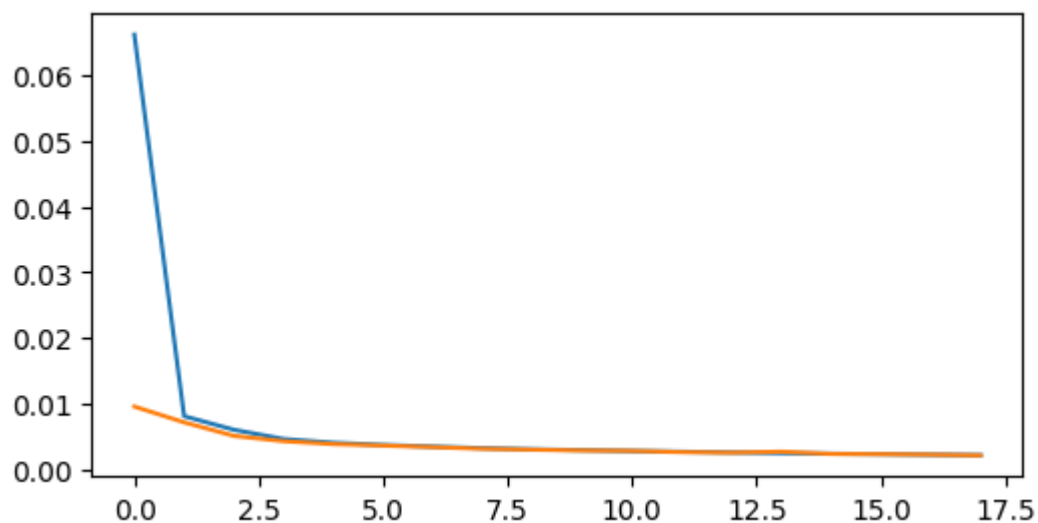
32%|███████| 16/50 [00:28<00:55, 1.64s/it]epoch:15, cur train loss: 0.00225227168



0 20 40 0 20 40
34% |███████| 17/50 [00:29<00:53, 1.63s/it]epoch:16, cur train loss: 0.00217570869

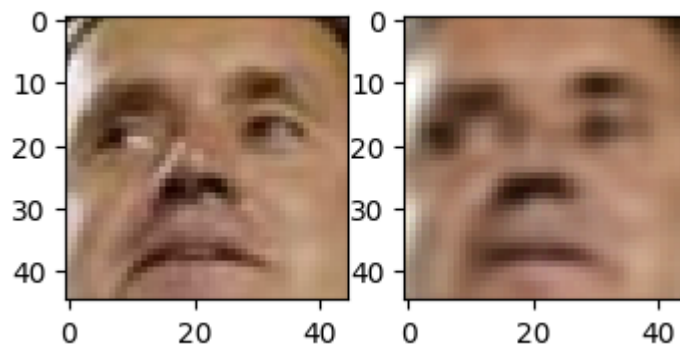
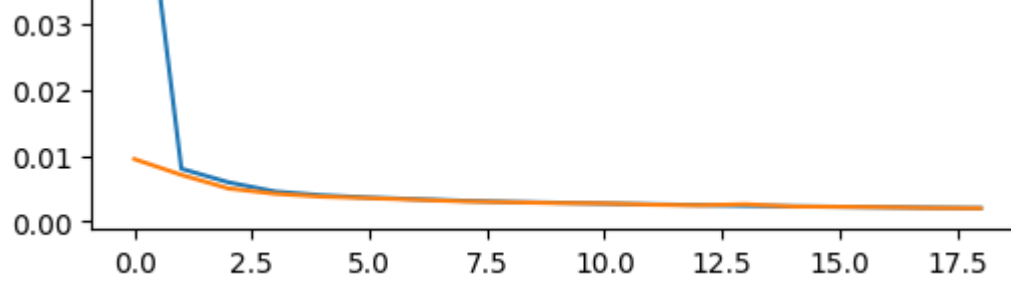


36% |███████| 18/50 [00:31<00:51, 1.62s/it]epoch:17, cur train loss: 0.002106086135

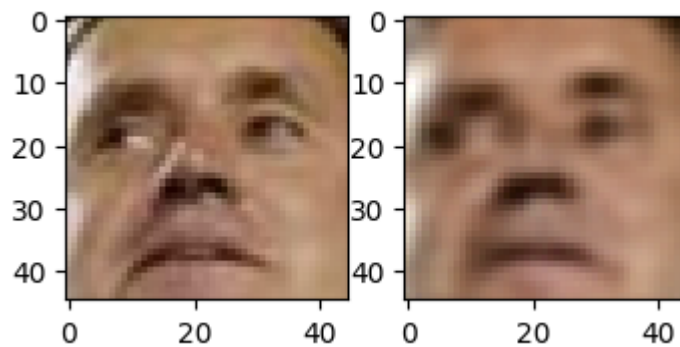
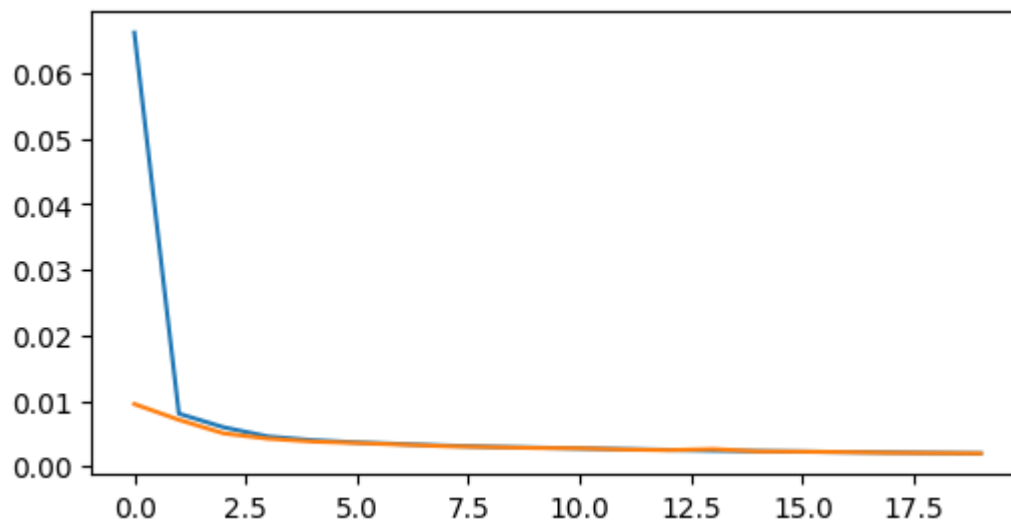


38% |███████| 19/50 [00:32<00:50, 1.63s/it]epoch:18, cur train loss: 0.00205026908

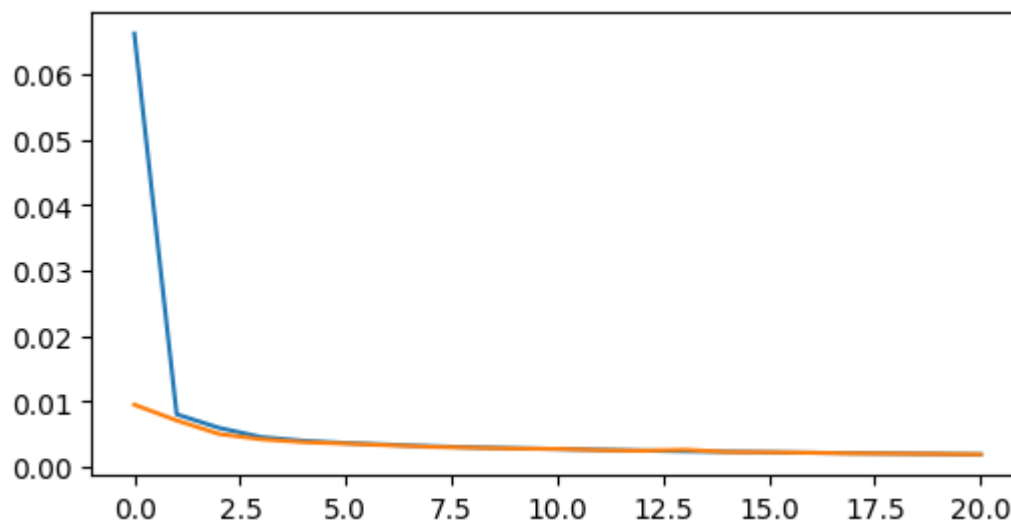


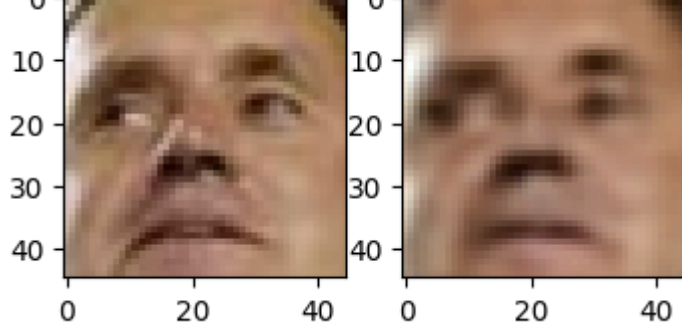


40% | ██████████ | 20/50 [00:34<00:48, 1.63s/it]epoch:19, cur train loss: 0.001989269236

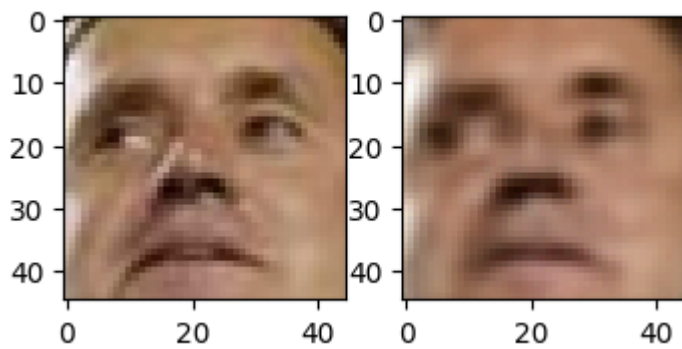
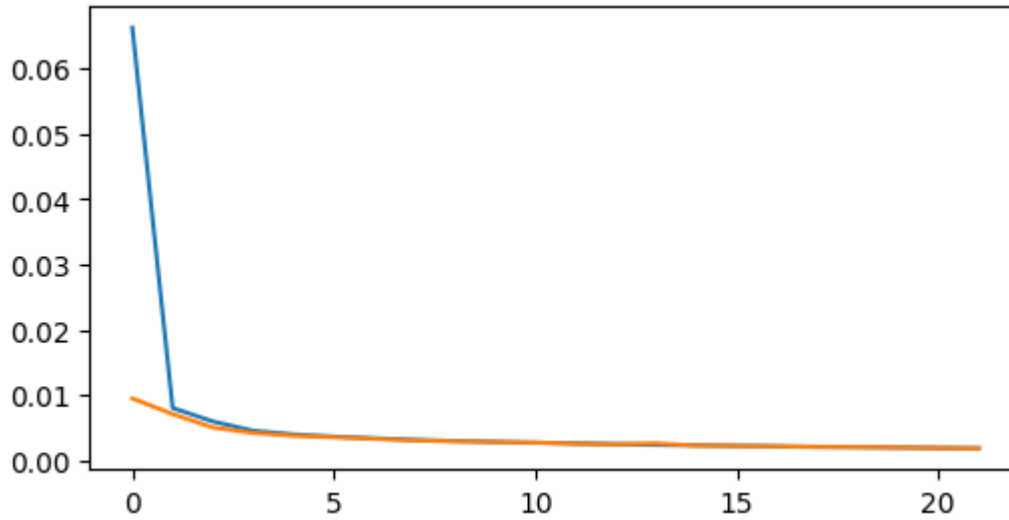


42% | ██████████ | 21/50 [00:36<00:47, 1.62s/it]epoch:20, cur train loss: 0.00192800885

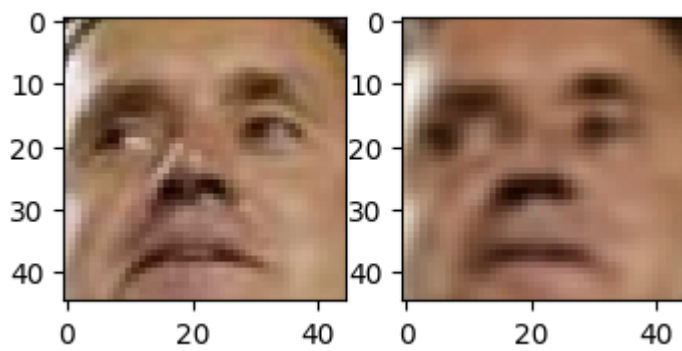
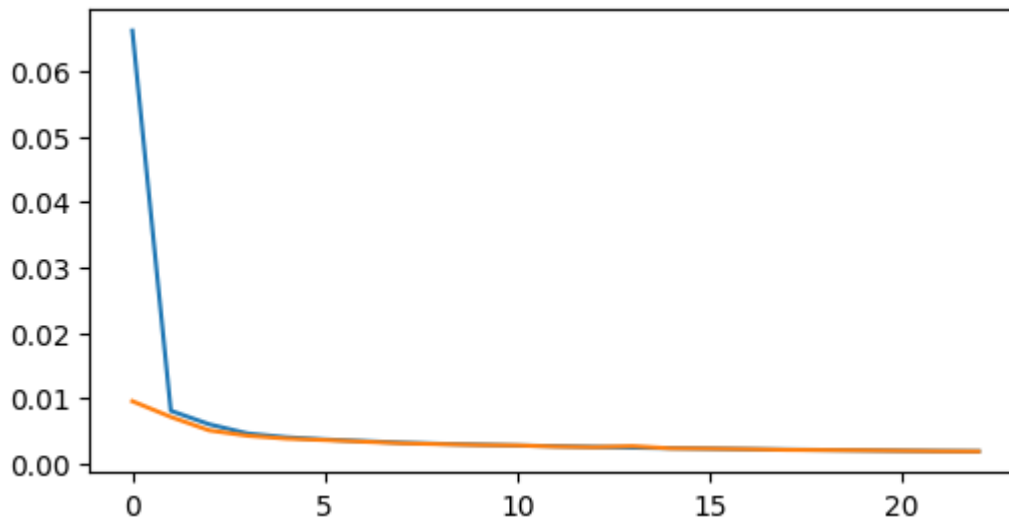




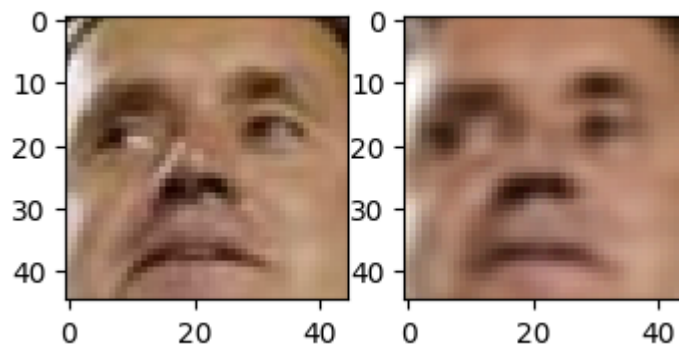
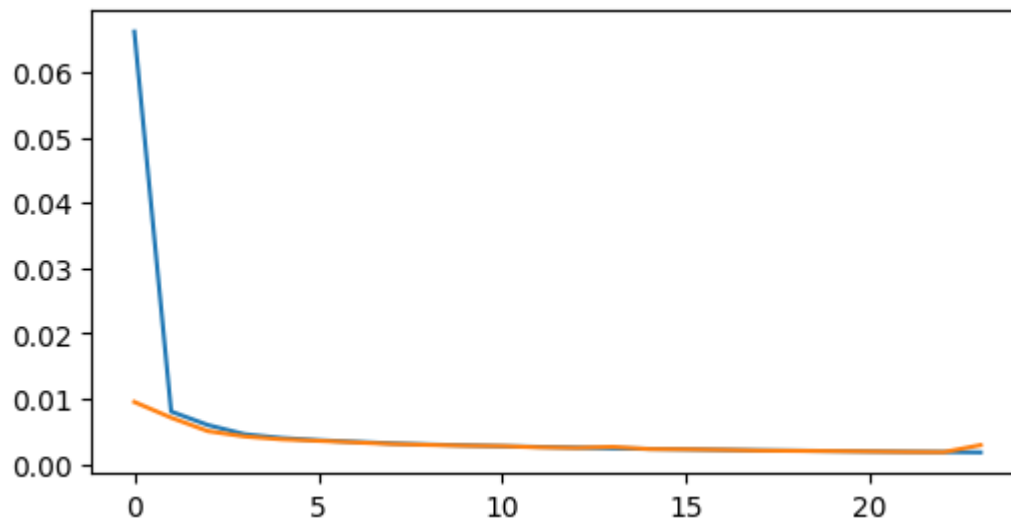
44%|███████| 22/50 [00:37<00:45, 1.62s/it]epoch:21, cur train loss: 0.00188320980



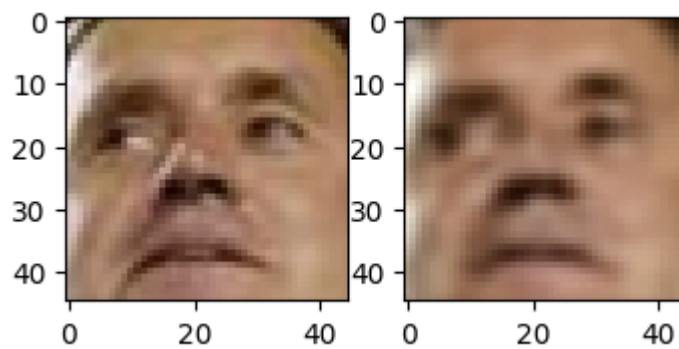
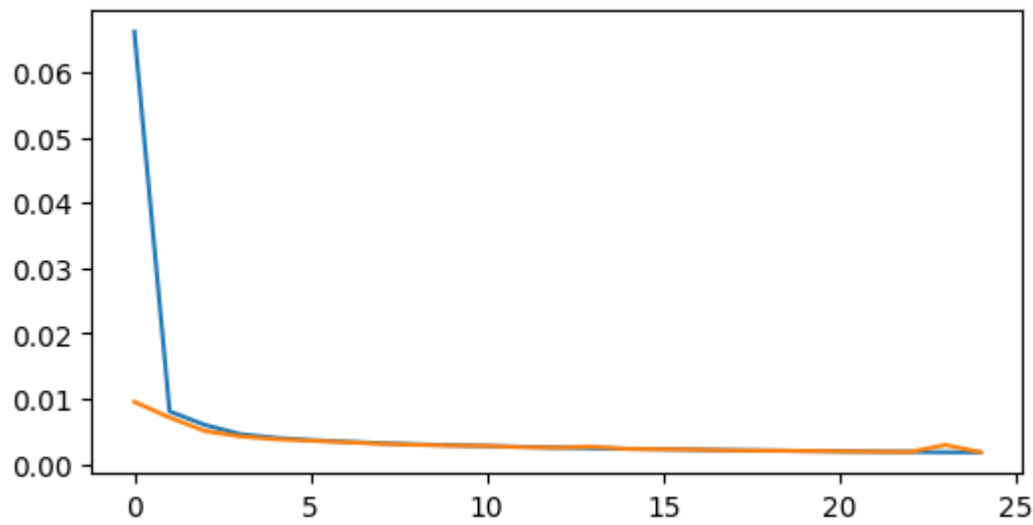
46%|███████| 23/50 [00:39<00:43, 1.62s/it]epoch:22, cur train loss: 0.001848052585



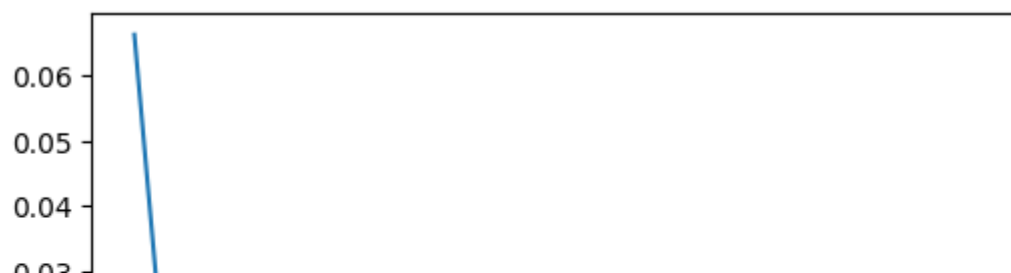
48%|██████| 24/50 [00:40<00:41, 1.61s/it]epoch:23, cur train loss: 0.00178969852

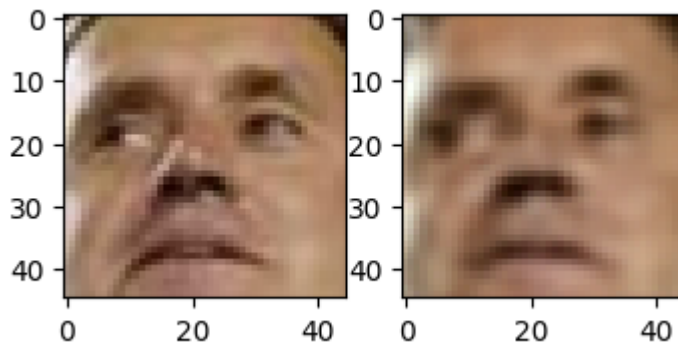
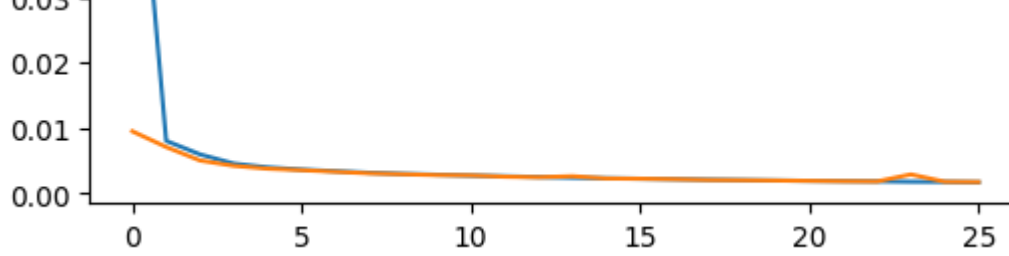


50%|██████| 25/50 [00:42<00:40, 1.62s/it]epoch:24, cur train loss: 0.001769431353

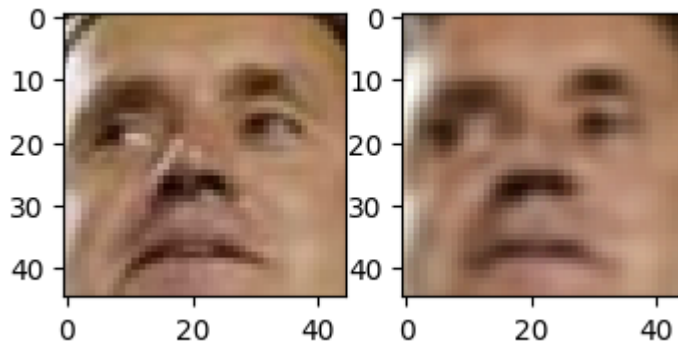
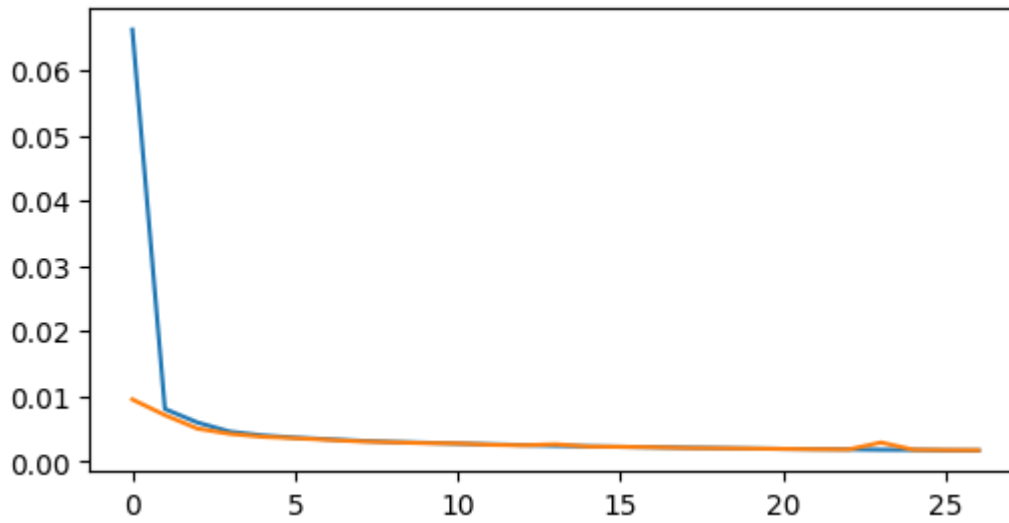


52%|██████| 26/50 [00:44<00:38, 1.62s/it]epoch:25, cur train loss: 0.00171658847

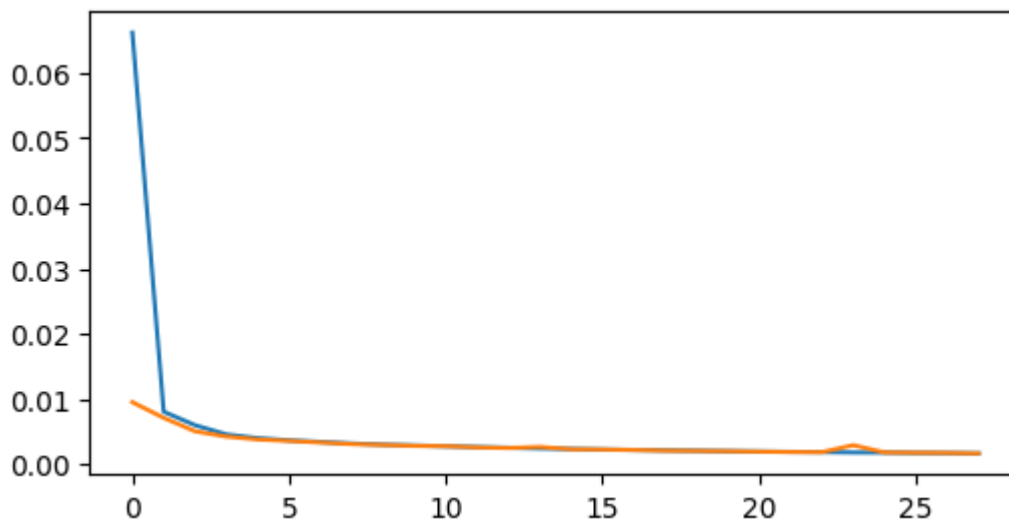


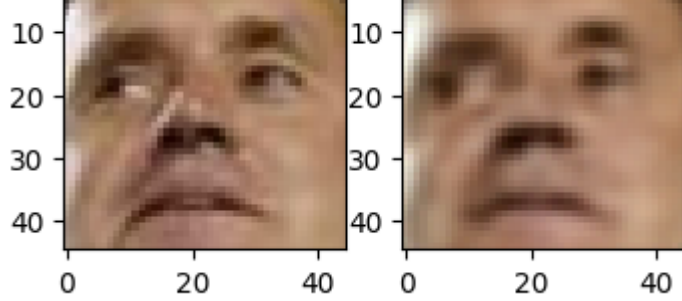


54%|██████| 27/50 [00:46<00:39, 1.70s/it]epoch:26, cur train loss: 0.00169768758

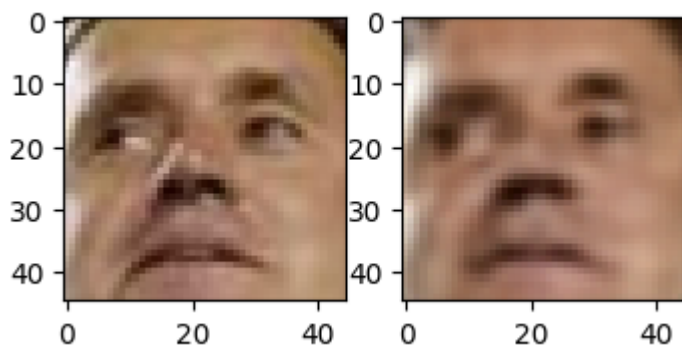
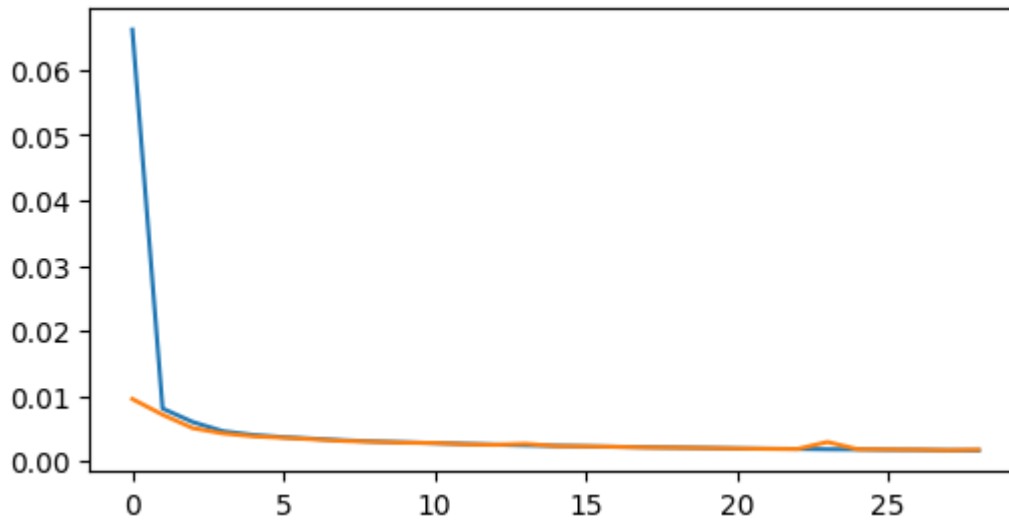


56%|██████| 28/50 [00:47<00:36, 1.68s/it]epoch:27, cur train loss: 0.001649529134

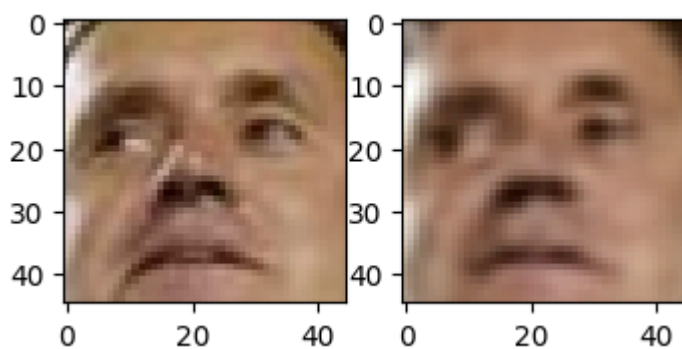
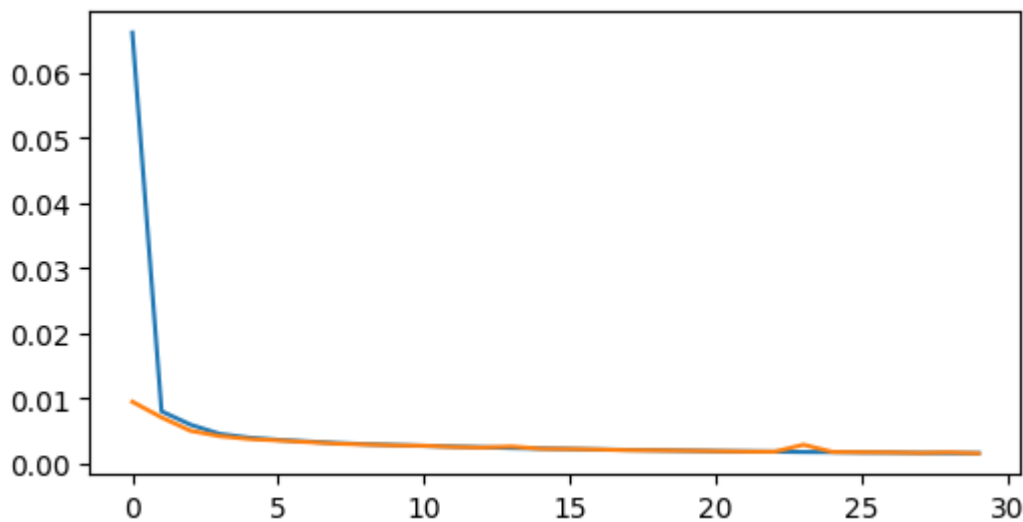




58%|███████| 29/50 [00:49<00:34, 1.67s/it]epoch:28, cur train loss: 0.00162392886

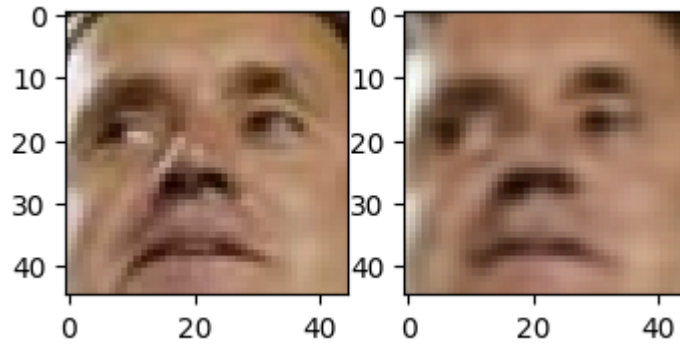
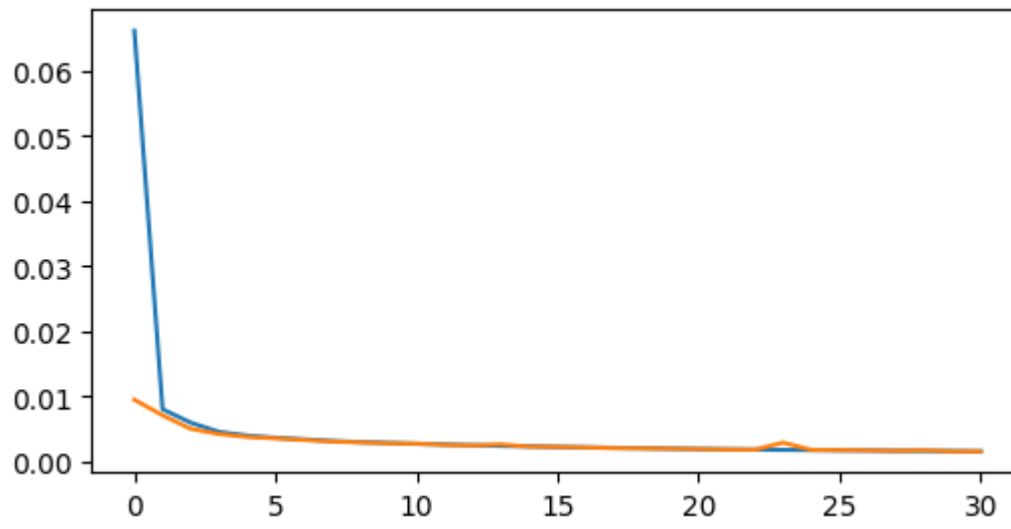


60%|███████| 30/50 [00:50<00:33, 1.66s/it]epoch:29, cur train loss: 0.001612258051

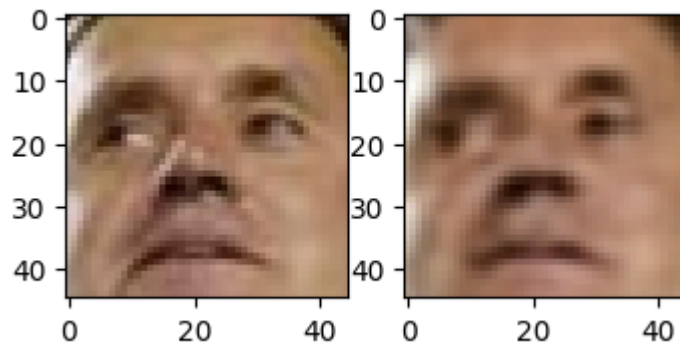
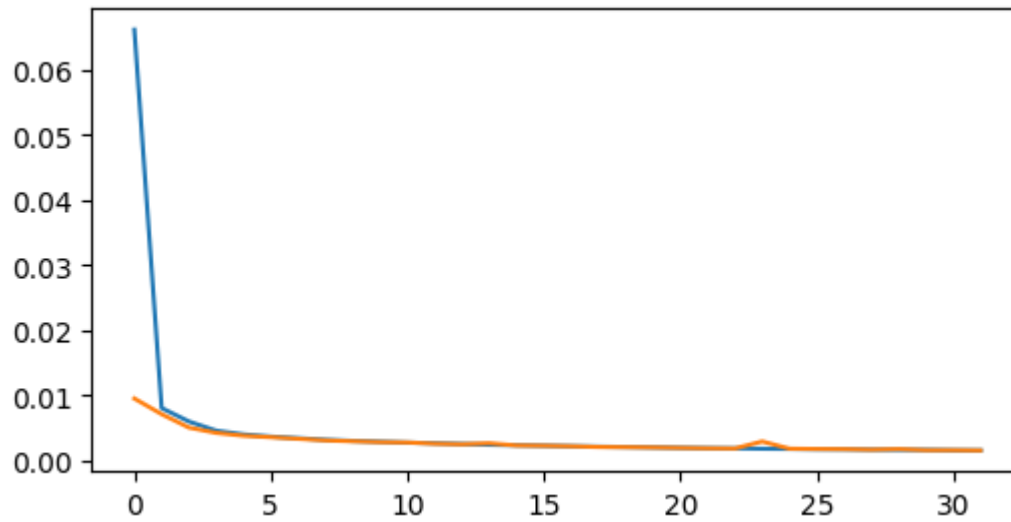


62%|███████| 31/50 [00:50<00:31, 1.66s/it]epoch:30, cur train loss: 0.00156326677

62%|██████████| 31/50 [00:52<00:31, 1.66s/it]epoch:30, cur train loss: 0.00156336677

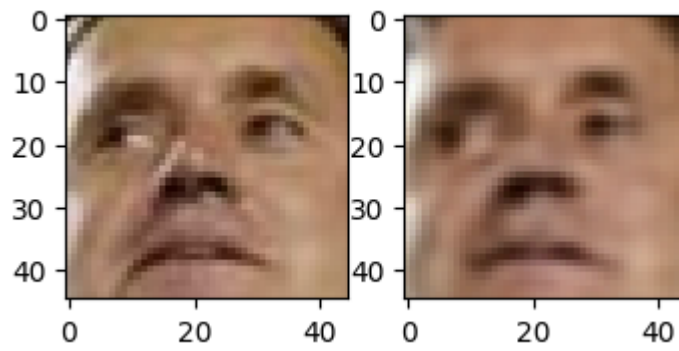
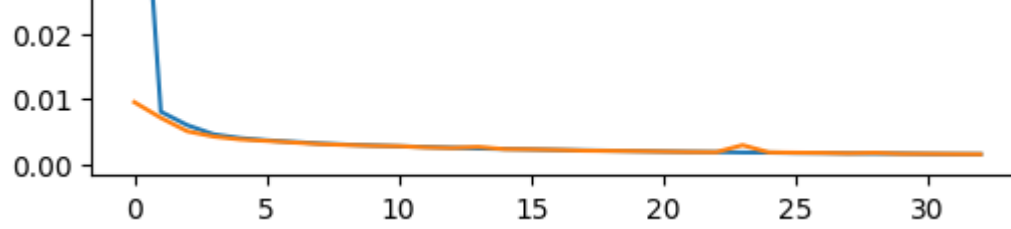


64%|██████████| 32/50 [00:54<00:29, 1.65s/it]epoch:31, cur train loss: 0.00154175839

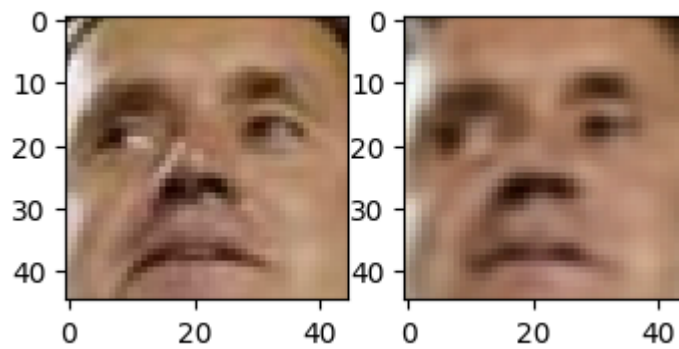
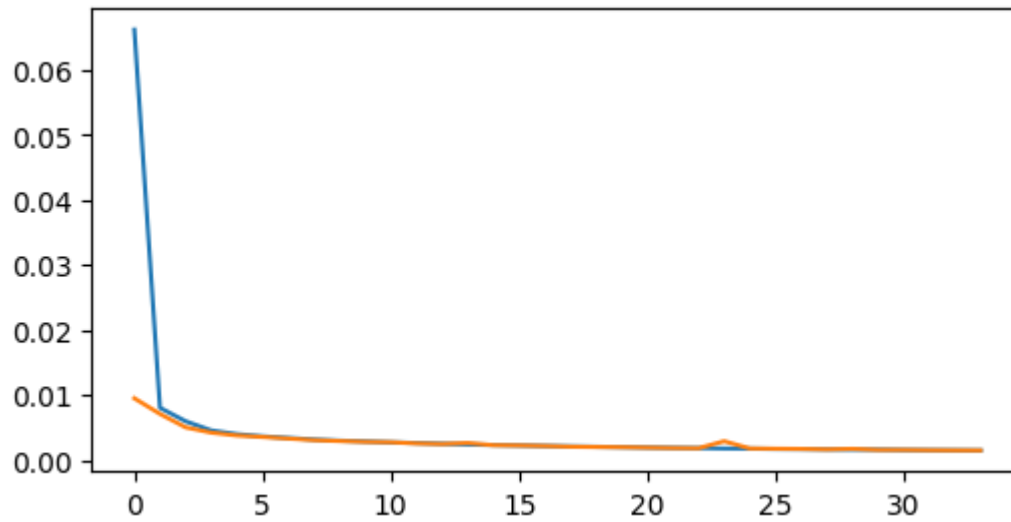


66%|██████████| 33/50 [00:55<00:27, 1.64s/it]epoch:32, cur train loss: 0.001522519562

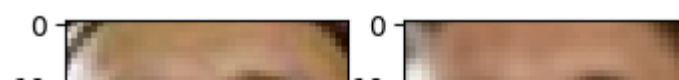
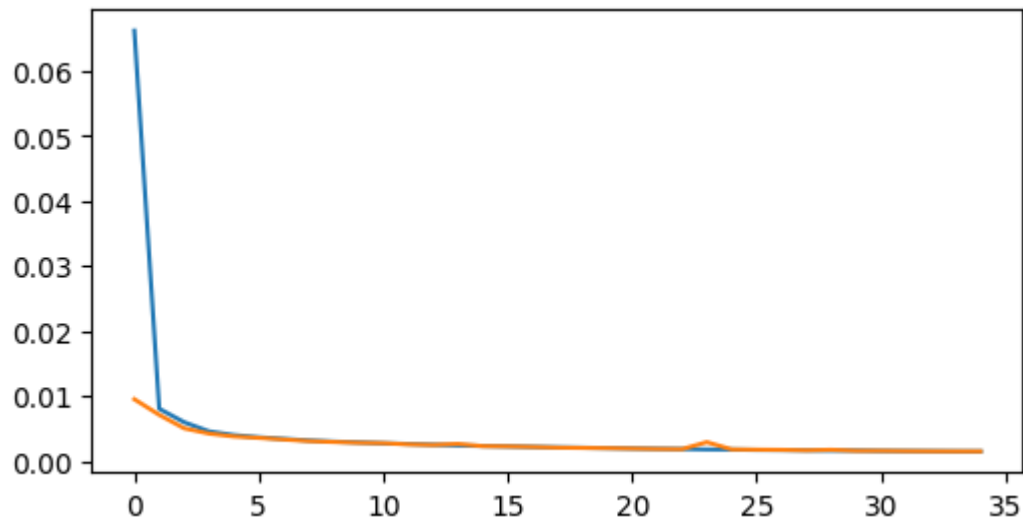


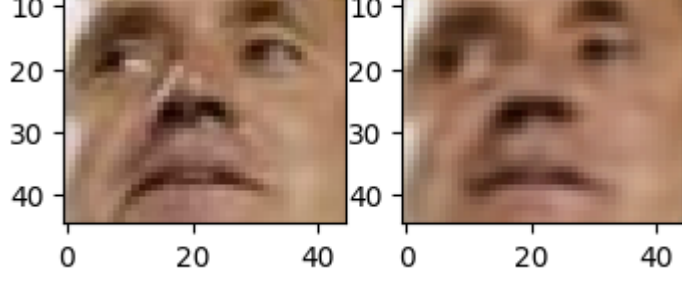


68%|███████ | 34/50 [00:57<00:26, 1.65s/it]epoch:33, cur train loss: 0.00149757913

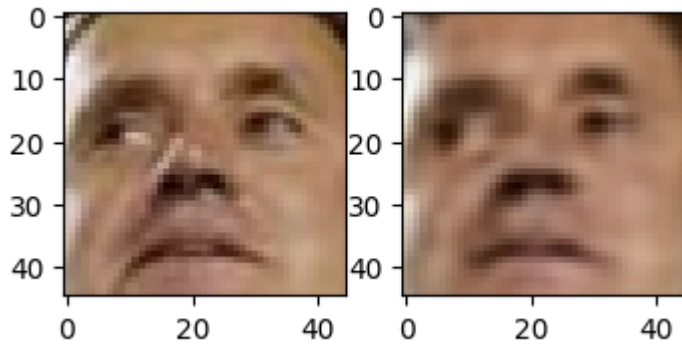
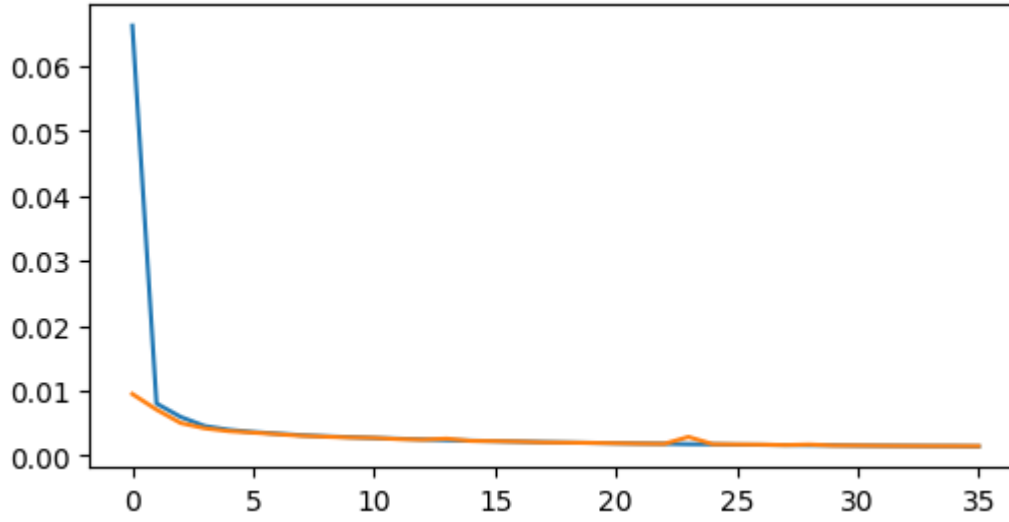


70%|███████ | 35/50 [00:59<00:24, 1.65s/it]epoch:34, cur train loss: 0.001487864977

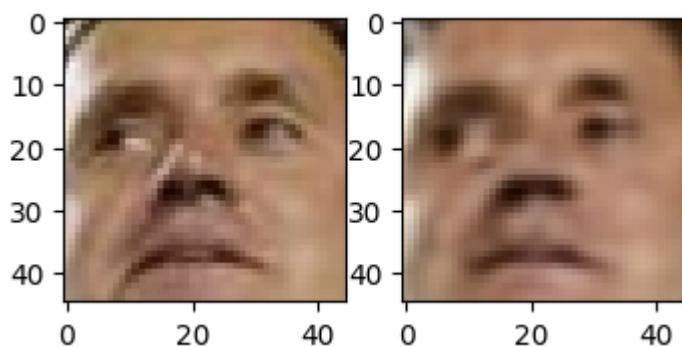
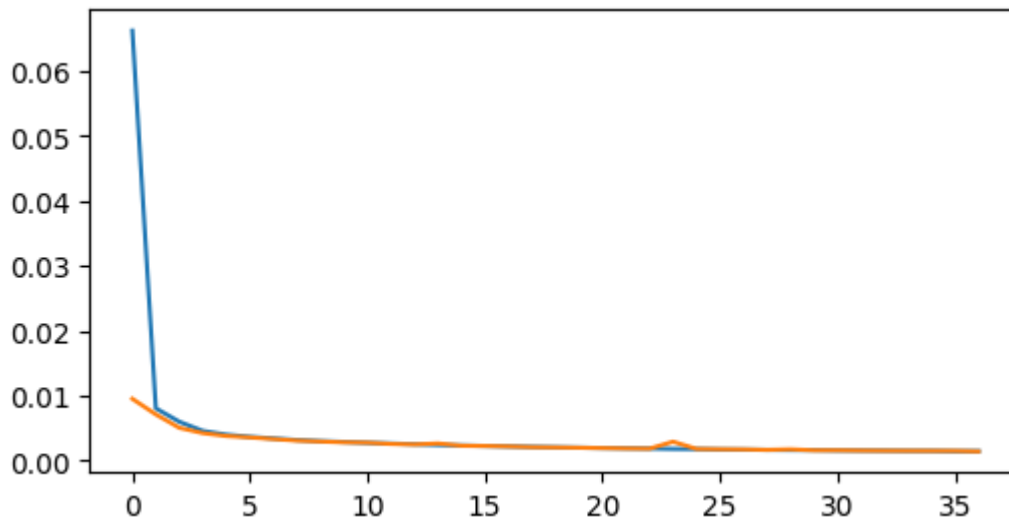




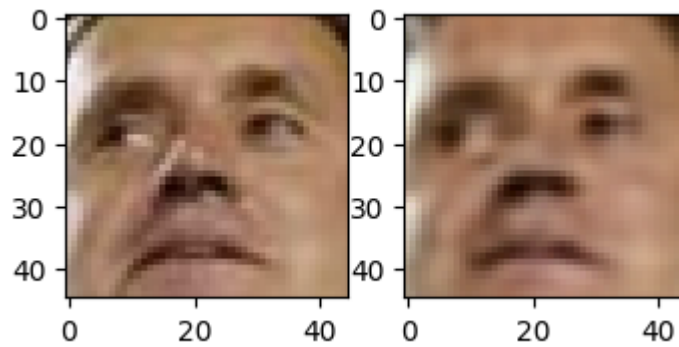
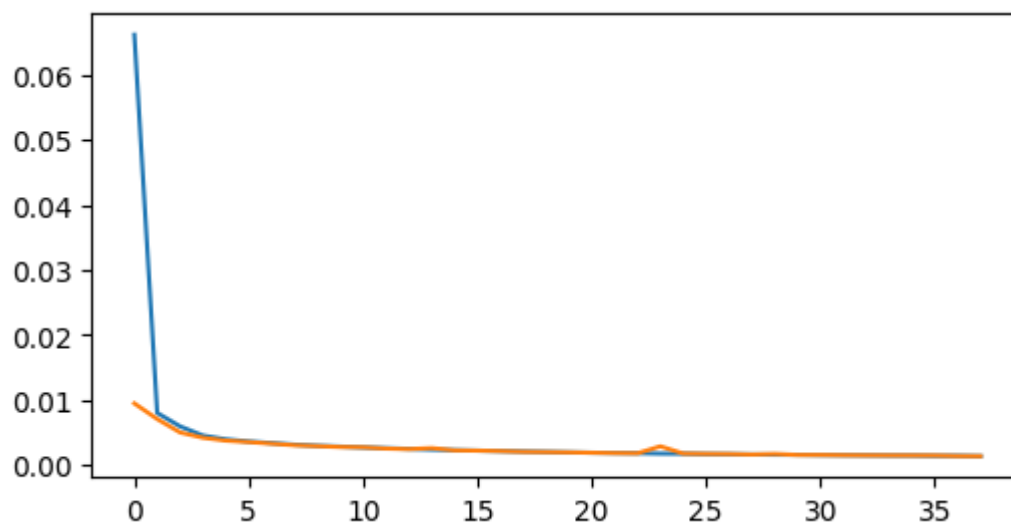
72%|██████████| 36/50 [01:00<00:23, 1.65s/it]epoch:35, cur train loss: 0.00144573527



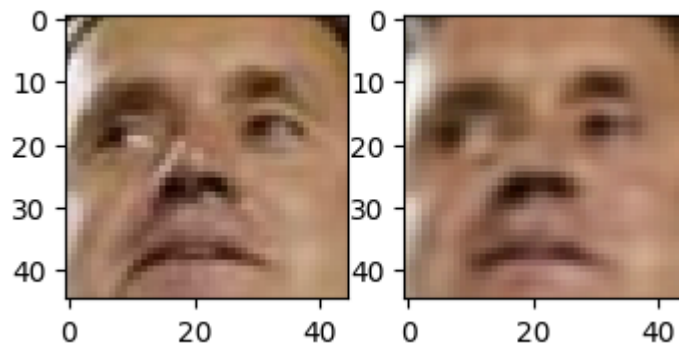
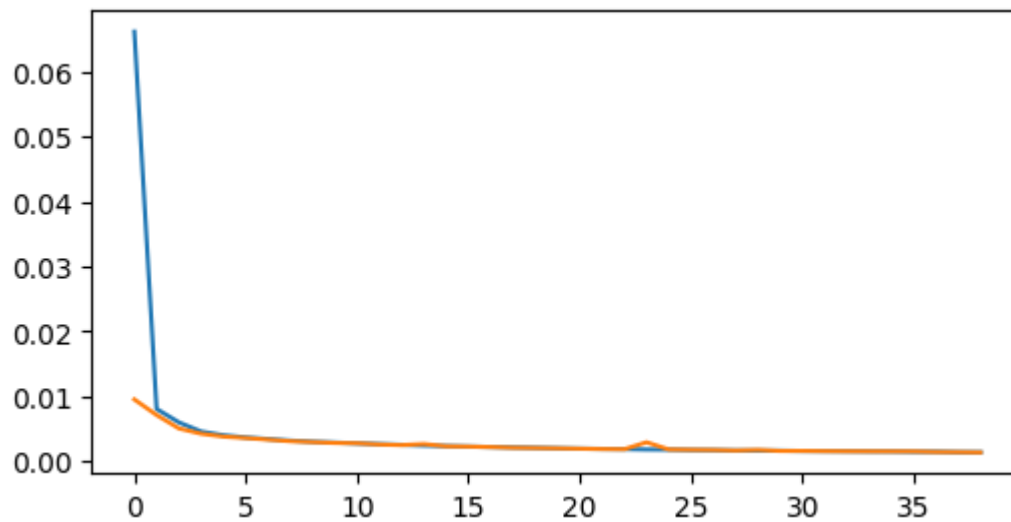
74%|██████████| 37/50 [01:02<00:21, 1.66s/it]epoch:36, cur train loss: 0.00144771971



76%|██████████| 38/50 [01:04<00:19, 1.65s/it]epoch:37, cur train loss: 0.001385573182

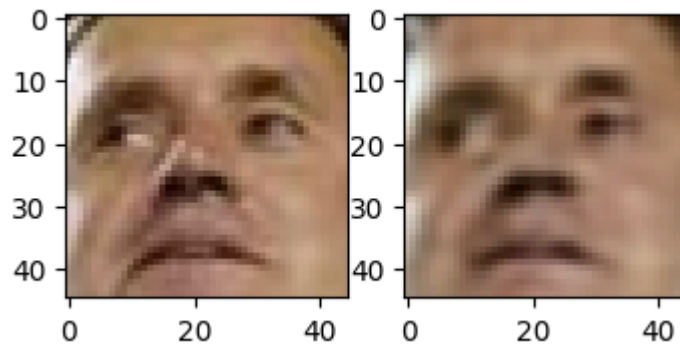
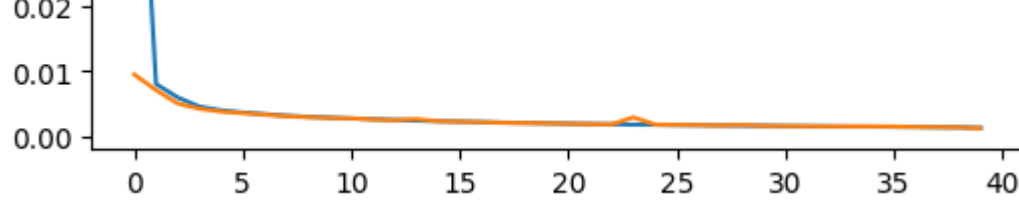


78%|██████████ | 39/50 [01:05<00:18, 1.65s/it]epoch:38, cur train loss: 0.00137404581

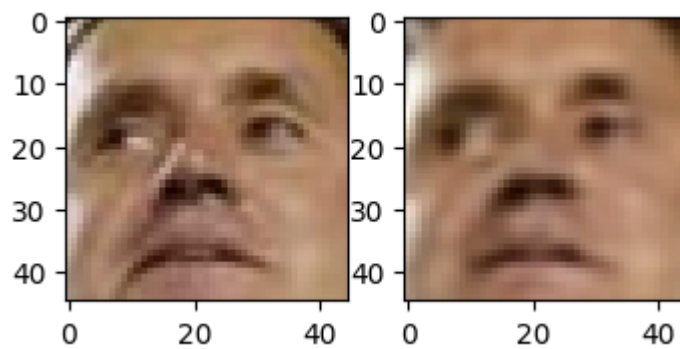
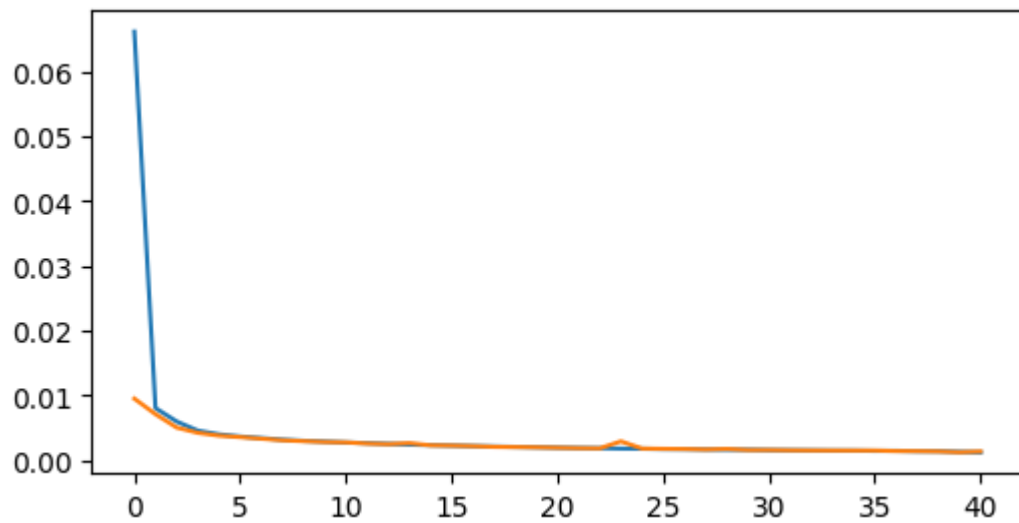


80%|██████████ | 40/50 [01:07<00:16, 1.65s/it]epoch:39, cur train loss: 0.001286328805

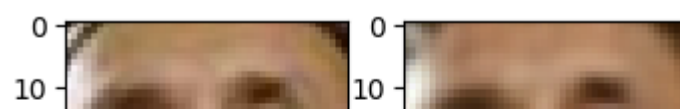
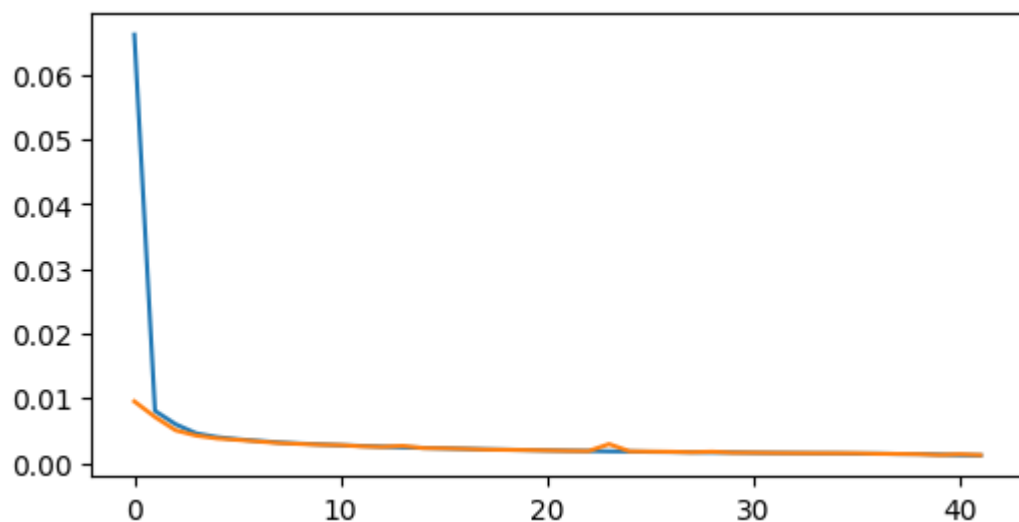


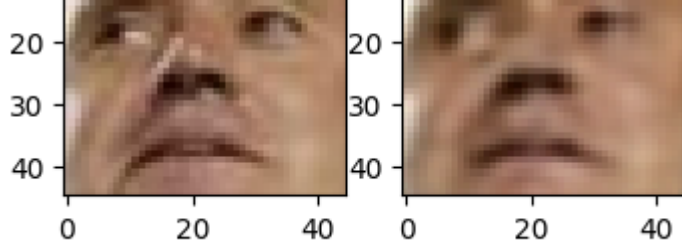


82% [██████████] | 41/50 [01:09<00:14, 1.65s/it]epoch:40, cur train loss: 0.00120403693

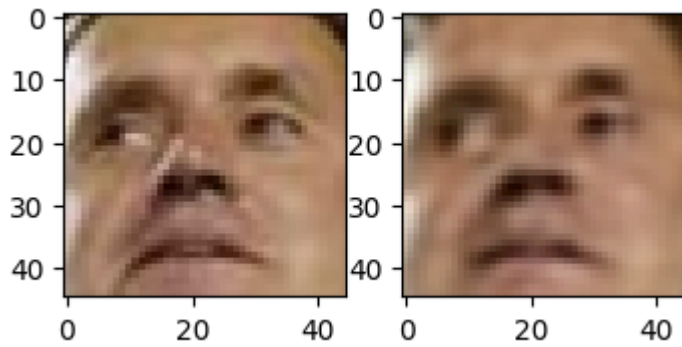
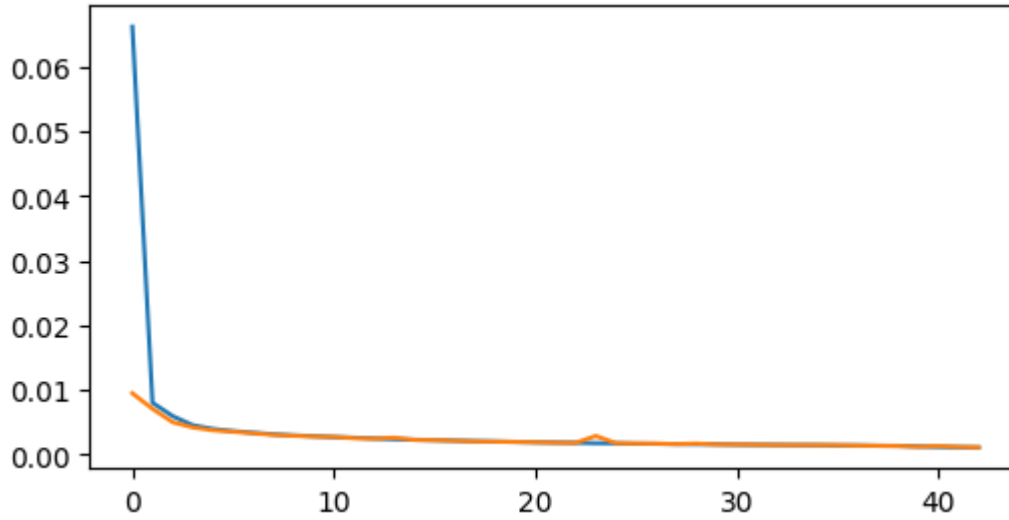


84% [██████████] | 42/50 [01:10<00:13, 1.72s/it]epoch:41, cur train loss: 0.00116488578

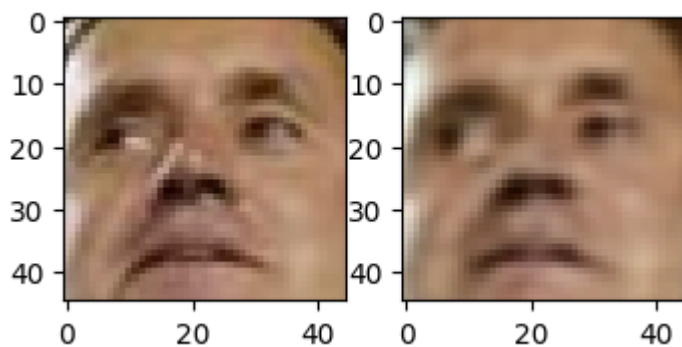
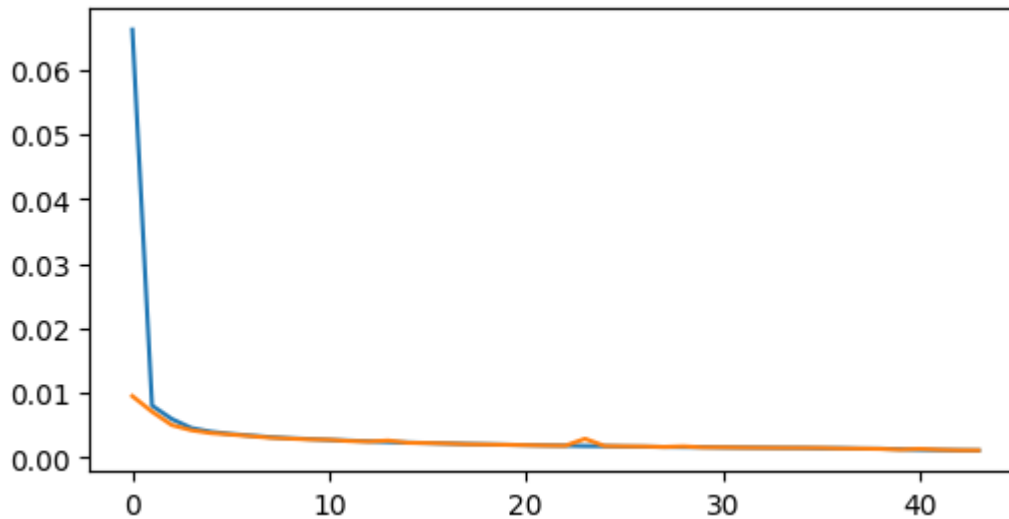




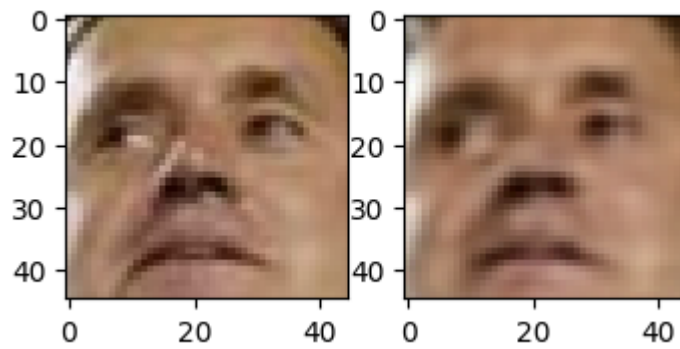
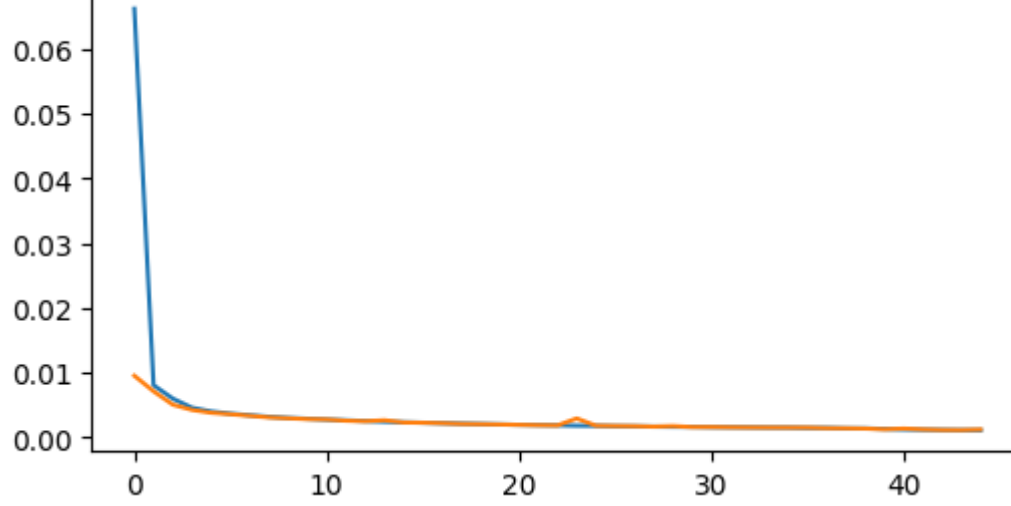
86%|██████████ | 43/50 [01:12<00:11, 1.71s/it]epoch:42, cur train loss: 0.001138708259



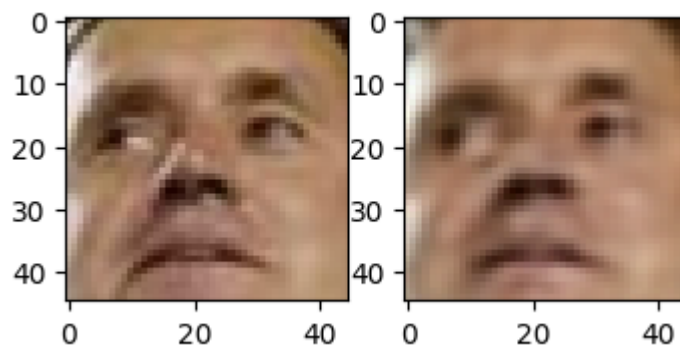
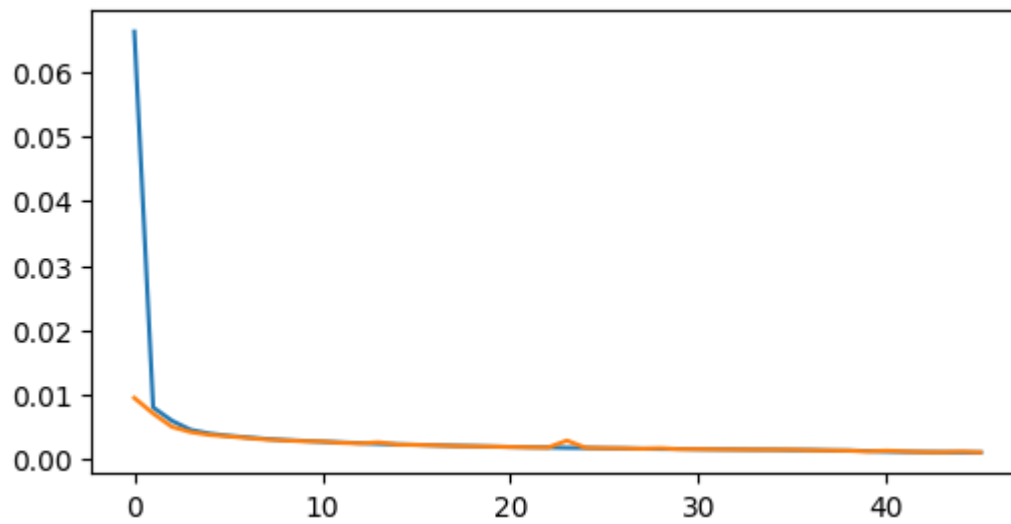
88%|██████████ | 44/50 [01:14<00:10, 1.69s/it]epoch:43, cur train loss: 0.00111339487



90%|██████████ | 45/50 [01:15<00:08, 1.68s/it]epoch:44, cur train loss: 0.001091231186

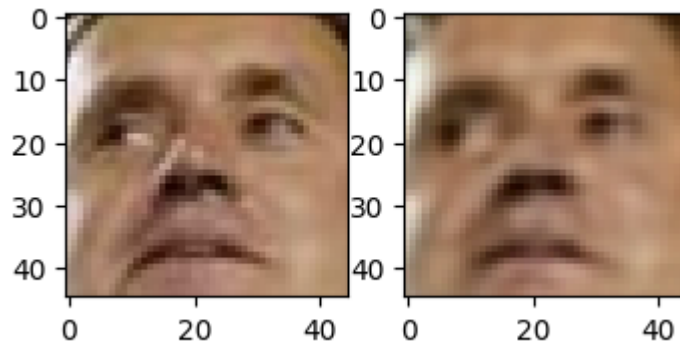
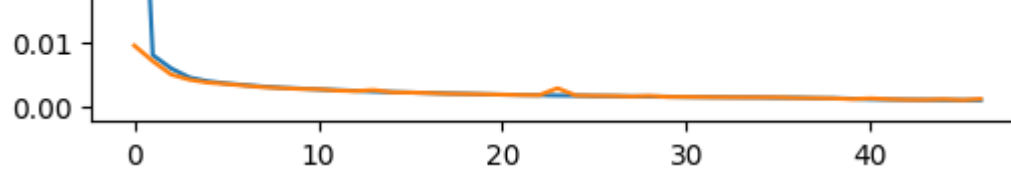


92% [██████████] | 46/50 [01:17<00:06, 1.67s/it]epoch:45, cur train loss: 0.00108364148

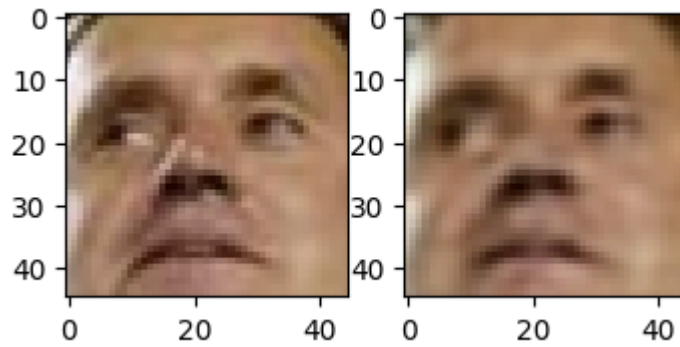
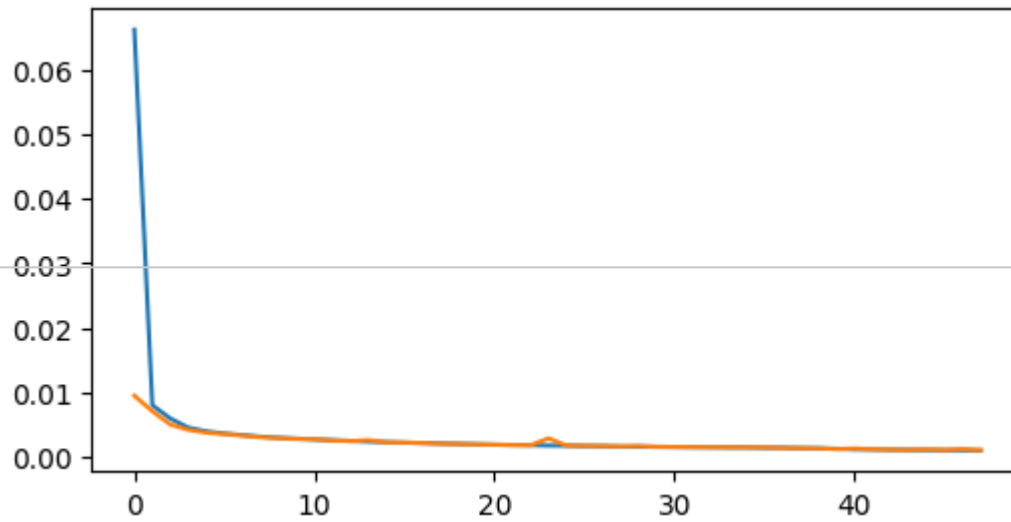


94% [██████████] | 47/50 [01:19<00:04, 1.67s/it]epoch:46, cur train loss: 0.00106089629

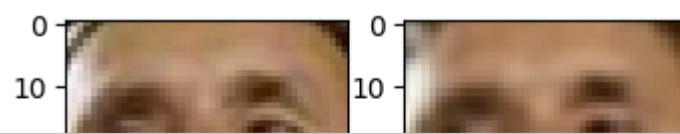
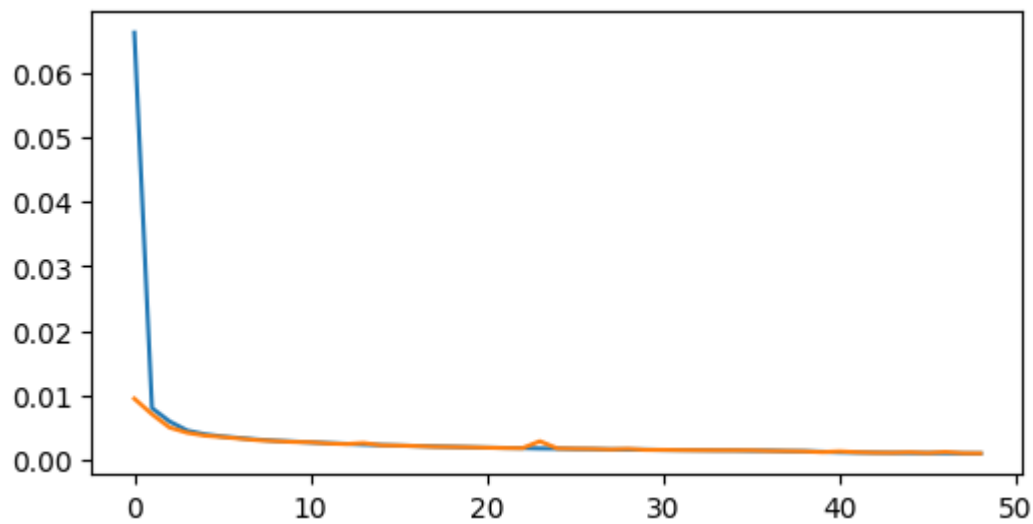




96% |██████████| 48/50 [01:20<00:03, 1.66s/it]epoch:47, cur train loss: 0.001041892272



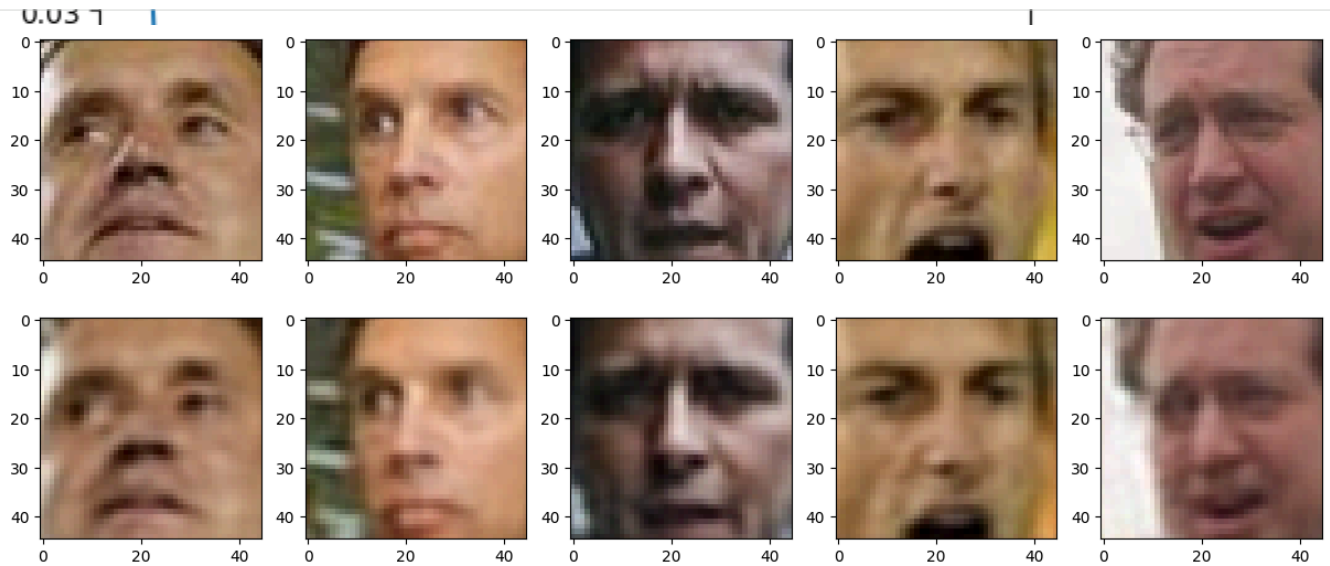
98% |██████████| 49/50 [01:22<00:01, 1.68s/it]epoch:48, cur train loss: 0.00103249788



Давайте посмотрим, как наш тренированный автоэнкодер кодирует и восстанавливает картинку:

```
model.eval()
val_subset = X_test_tensor[:5].to(device)
with torch.no_grad():
    recon_subset, _ = model(val_subset)
fig, axes = plt.subplots(2, 5, figsize=(15, 6))
for i in range(5):
    axes[0, i].imshow(val_subset[i].cpu().permute(1, 2, 0).clamp(0, 1))
    axes[1, i].imshow(recon_subset[i].cpu().permute(1, 2, 0).clamp(0, 1))

plt.show()
```



Что вы можете сказать про результат?

✓ Результат хороший #учитывая что мы полностью разрушаем пространственную информацию

› 1.4. Sampling (2 балла)

↳ Скрыто 2 ячейки.

✓ 1.5 Time to make fun! (3 балла)

Давайте научимся пририсовывать людям улыбки =)

so linear

this is you when looking at the HW for the first time



this is you after all

План такой:

1. Нужно выделить "вектор улыбки": для этого нужно из выборки изображений найти несколько (~15) людей с улыбками и столько же без.

Найти людей с улыбками вам поможет файл с атрибутами, скачанный вместе с картинками. В нем указаны имена изображений и присутствующие атрибуты (улыбки, очки и прочее интересное).

2. Вычислить латентный вектор для всех улыбающихся людей (прогнать их через encoder) и то же для всех грустных.
3. Вычислить, собственно, вектор улыбки -- посчитать разность между средним латентным вектором улыбающихся людей и средним латентным вектором грустных людей.
4. А теперь приделаем улыбку грустному человеку: добавим полученный в пункте 3 вектор к латентному вектору грустного человека и прогоним полученный вектор через decoder. Получим того же человека, но уже не грустного!

```
full_images = torch.cat([X_train_tensor, X_test_tensor], dim=0)
full_attrs = pd.concat([y_train, y_test], ignore_index=True)
```

```
smile_col = 'Smiling'
sorted_attrs = full_attrs.sort_values(smile_col)
fun_idx = sorted_attrs.tail(20).index
sad_idx = sorted_attrs.head(20).index
```



```
fun_batch = full_images[fun_idx].to(device)    #
sad_batch = full_images[sad_idx].to(device)
```

```
sad_idx
```

```
Index([ 6453,  5451, 11590,  9530,  8222, 10646,  1127,  1361,  5315, 11691,
        3755,  3837,  9275, 11395,  6111,  2958, 10140,  5196, 10825,  7562],
      dtype='int64')
```

```
with torch.no_grad():
    emb_fun = model.enc(fun_batch)
    emb_sad = model.enc(sad_batch)

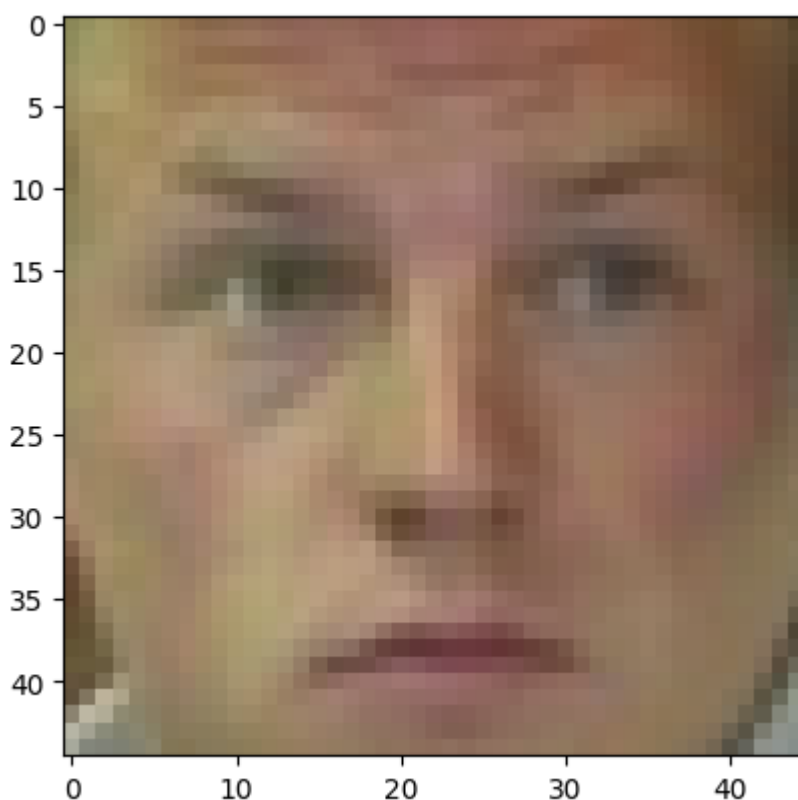
mean_fun = emb_fun.mean(dim=0)
mean_sad = emb_sad.mean(dim=0)
smile_vector = mean_fun - mean_sad
```

```
idx = sad_idx[2]
img_sad = full_images[idx].unsqueeze(0).to(device)
```

```
with torch.no_grad():
    _, latent_sad = model(img_sad)
    latent_smiled = latent_sad + smile_vector
    img_smiled = model.dec(latent_smiled)
```

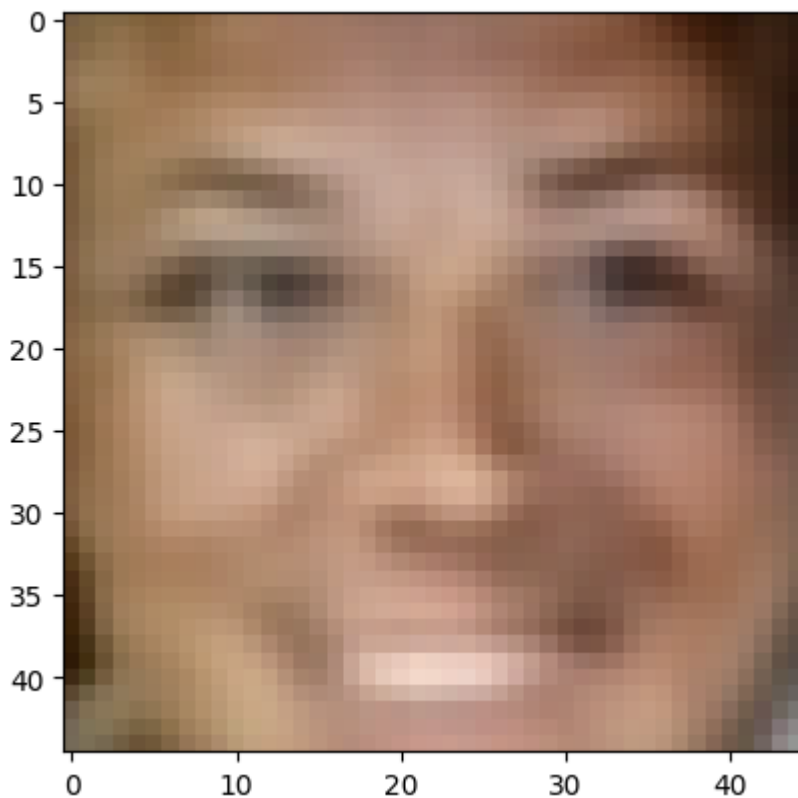
```
plt.imshow(img_sad[0].cpu().permute(1, 2, 0).clamp(0, 1).numpy())
```

```
<matplotlib.image.AxesImage at 0x7d1c8a05cf50>
```



```
plt.imshow(img_smiled[0].cpu().permute(1, 2, 0).clamp(0, 1).numpy())
```

<matplotlib.image.AxesImage at 0x7d1c89fa2f30>



Вуаля! Вы восхитительны!

Теперь вы можете пририсовывать людям не только улыбки, но и много чего другого -- закрывать/открывать глаза, пририсовывать очки... В общем, все, на что хватит фантазии и на что есть атрибуты в `attrs` :)

› Часть 2: Variational Autoencoder (10 баллов)

↳ Скрыта 21 ячейка.

✓ Видно чёткое разделение латентных представлений на группы по классам

Congrats v2.0!

✓ 2.3. Conditional VAE (6 баллов)

Мы уже научились обучать обычный АЕ на датасете картинок и получать новые картинки, используя генерацию шума и декодер. Давайте теперь допустим, что мы обучили АЕ на датасете MNIST и теперь хотим генерировать новые картинки с числами с помощью

декодера (как выше мы генерили рандомные лица). И вот нам понадобилось сгенерировать цифру 8, и мы подставляем разные варианты шума, но восьмерка никак не генерится:(

Хотелось бы добавить к нашему АЕ функцию "выдай мне рандомное число из вот этого вот класса", где классов десять (цифры от 0 до 9 образуют десять классов). Conditional AE — так называется вид автоэнкодера, который предоставляет такую возможность. Ну, название "conditional" уже говорит само за себя.

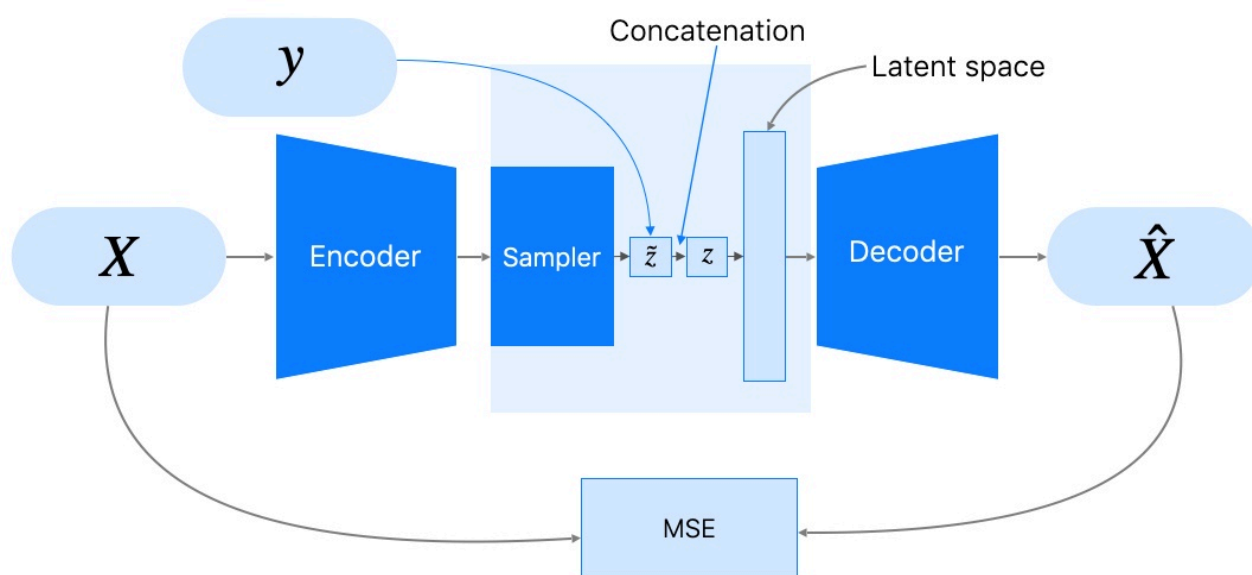
И в этой части задания мы научимся такие обучать.

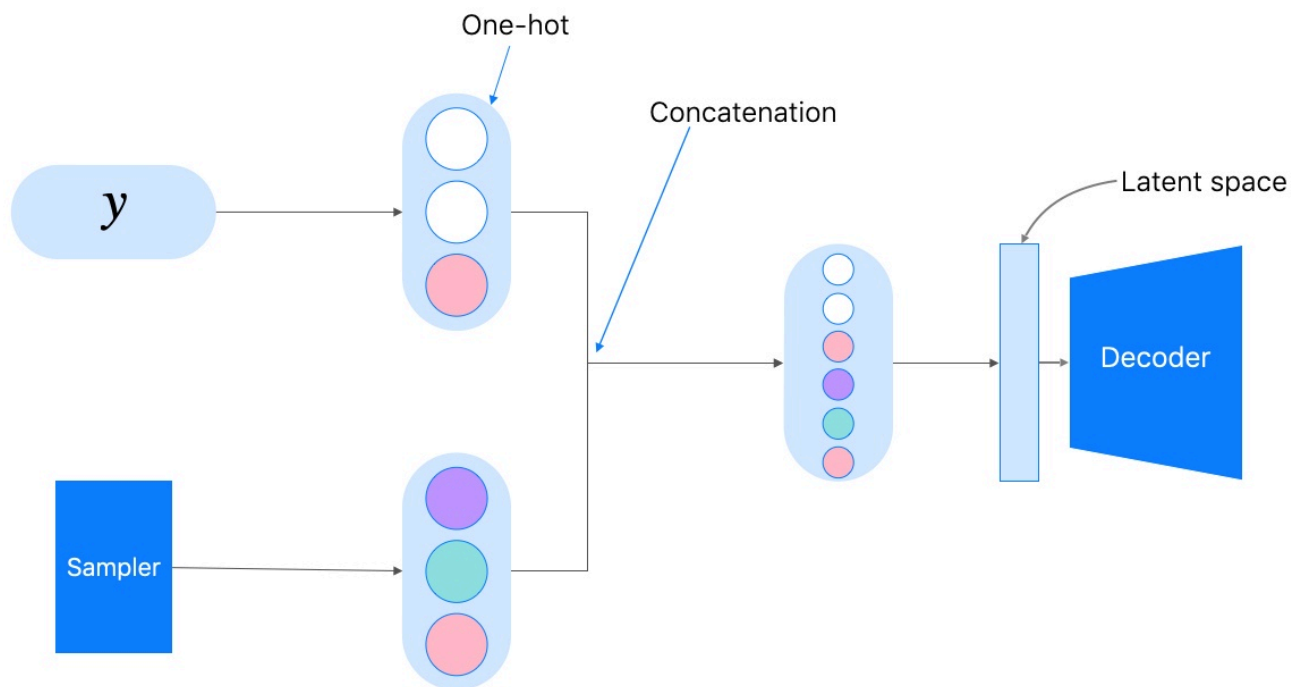
✓ Архитектура

На картинке ниже представлена архитектура простого Conditional VAE.

По сути, единственное отличие от обычного -- это то, что мы вместе с картинкой в первом слое энкодера и декодера передаем еще информацию о классе картинки.

То есть, в первый (входной) слой энкодера подается конкатенация картинки и информации о классе (например, вектора из девяти нулей и одной единицы). В первый слой декодера подается конкатенация латентного вектора и информации о классе.





На всякий случай: это VAE, то есть, latent у него все еще состоит из μ и σ

Таким образом, при генерации новой случайной картинки мы должны будем передать декодеру сконкатенированные латентный вектор и класс картинки.

P.S. Также можно передавать класс картинки не только в первый слой, но и в каждый слой сети. То есть на каждом слое конкатенировать выход из предыдущего слоя и информацию о классе.

```
class CVAE(nn.Module):
    def __init__(self, latent_dim = 128, num_classes = 10):
        super(CVAE, self).__init__()

        self.num_classes = num_classes

        self.enc = nn.Sequential(
            nn.Conv2d(1 + num_classes, 32, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 512),
            nn.ReLU()
        )
        self.fc_mu = nn.Linear(512, latent_dim)
        self.fc_logsigma = nn.Linear(512, latent_dim)

        self.decoder_input = nn.Sequential(
            nn.Linear(latent_dim + num_classes, 512),
            nn.ReLU()
        )
        self.dec = nn.Sequential(
            nn.Linear(512, 64 * 7 * 7),
```

```

        nn.ReLU(),
        nn.Unflatten(1, (64, 7, 7)),
        nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1, output_padding=1),
        nn.ReLU(),
        nn.ConvTranspose2d(32, 1, 3, stride=2, padding=1, output_padding=1),
        nn.Sigmoid()
    )

def onehot(self, class_num):
    onehot = F.one_hot(class_num, num_classes=self.num_classes).float()
    onehot = onehot.view(-1, self.num_classes, 1, 1).expand(-1, -1, 28, 28)
    return onehot

def encode(self, x, class_num):
    onehot = self.onehot(class_num)
    x = torch.cat([x, onehot], dim=1)
    x = self.enc(x)
    mu = self.fc_mu(x)
    logsigma = self.fc_logsigma(x)
    return mu, logsigma, onehot

def gaussian_sampler(self, mu, logsigma):
    if self.training:
        sigma = torch.exp(0.5*logsigma)#####

        eps = torch.randn_like(sigma)
        return mu + eps * sigma

    else:
        # на инференсе возвращаем не случайный вектор из нормального распределения
        # на инференсе выход автоэнкодера должен быть детерминирован.
        return mu

def decode(self, z, onehot):
    class_vec = onehot.mean(dim=[2, 3])
    z = torch.cat([z, class_vec], dim=1)
    reconstruction = self.decoder_input(z)
    reconstruction = self.dec(reconstruction)
    return reconstruction

def forward(self, x, class_num):
    mu, logsigma, onehot = self.encode(x, class_num)
    z = self.gaussian_sampler(mu, logsigma)
    reconstruction = self.decode(z, onehot)
    return mu, logsigma, reconstruction

```

```

batch_size = 128
# MNIST Dataset
train_dataset = datasets.MNIST(root='./mnist_data/', train=True, transform=transforms.
test_dataset = datasets.MNIST(root='./mnist_data/', train=False, transform=transforms.

# Data Loader (Input Pipeline)

```

```
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=batch_size,
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=batch_size,
```

```
criterion = loss_vae
```

```
model = CVAE().to(device)
```

```
optimizer = optim.Adam(model.parameters(), lr = 3e-4 )
```

```
from tqdm import tqdm
import numpy as np
import matplotlib.pyplot as plt

epoch = 50
train_loss = []
test_loss = []

val_img, val_label = train_dataset[0]
val_vis = val_img.unsqueeze(0).to(device)
val_label = torch.tensor([val_label]).to(device)

for i in tqdm(range(epoch)):
    model.train()
    epoch_loss = {'train': [], 'test': []}

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        mu, logsigma, recon = model(images, labels)
        loss = criterion(images, mu, logsigma, recon)
        loss.backward()
        optimizer.step()
        epoch_loss['train'].append(loss.item())

    train_loss.append(np.mean(epoch_loss['train']))

    model.eval()
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
        with torch.no_grad():
            mu, logsigma, recon = model(images, labels)
            loss = criterion(images, mu, logsigma, recon)
            epoch_loss['test'].append(loss.item())

    test_loss.append(np.mean(epoch_loss['test']))

    with torch.no_grad():
        _, _, recon_vis = model(val_vis, val_label)

fig, axes = plt.subplots(1, 2, figsize=(4, 2))
axes[0].imshow(val_vis[0].cpu().squeeze())
axes[1].imshow(recon_vis[0].cpu().squeeze())
```

```
plt.show()

plt.figure(figsize=(6, 3))
plt.plot(train_loss, label='train')
plt.plot(test_loss, label='val')

plt.show()

print(f'epoch:{i}, cur train loss: {train_loss[-1]:.4f}, curr test loss: {test_loss[-1]:.4f}')
```

[Показать скрытые выходные данные](#)

✓ Sampling

Тут мы будем сэмплировать из CVAE. Это прикольное, чем сэмплировать из простого AE/VAE: тут можно взять один и тот же латентный вектор и попросить CVAE восстановить из него картинки разных классов! Для MNIST вы можете попросить CVAE восстановить из одного латентного вектора, например, картинки цифры 5 и 7.

```
import torch
import torch.nn.functional as F
import matplotlib.pyplot as plt

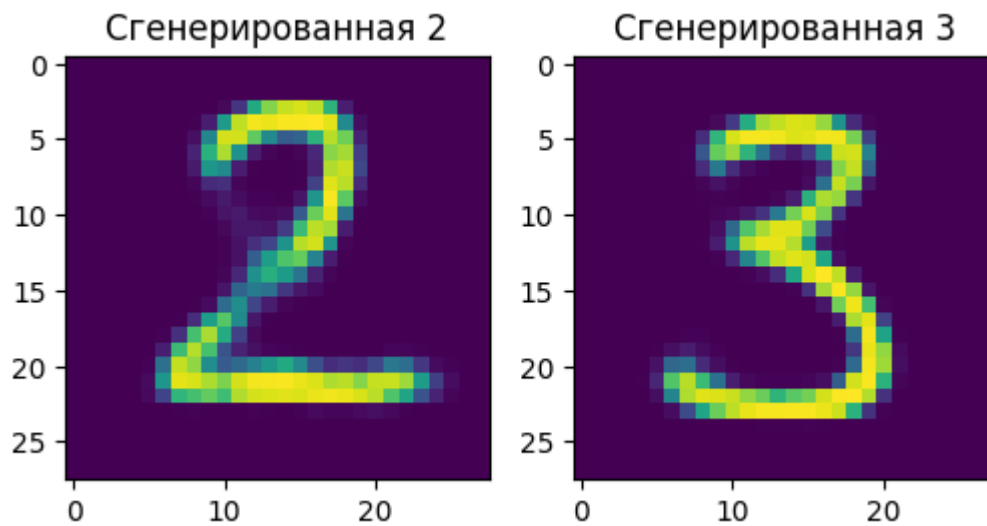
latent_dim = 128
z = torch.randn(1, latent_dim).to(device)

def generate_digit(model, z, class_idx):
    num_classes = model.num_classes
    onehot = F.one_hot(torch.tensor([class_idx]), num_classes=num_classes).float()
    onehot = onehot.view(1, num_classes, 1, 1).expand(-1, -1, 28, 28).to(device)
    with torch.no_grad():
        reconstruction = model.decode(z, onehot)
    return reconstruction.squeeze().cpu().numpy()

# Пример: генерируем 5 и 7
digit_5 = generate_digit(model, z, 2)
digit_7 = generate_digit(model, z, 3)

# Визуализация
fig, axes = plt.subplots(1, 2, figsize=(6, 3))
axes[0].imshow(digit_5)
axes[0].set_title('Сгенерированная 2')
axes[1].imshow(digit_7)
axes[1].set_title('Сгенерированная 3')

plt.show()
```



Splendid! Вы великолепны!

✓ Latent Representations

Давайте посмотрим, как выглядит латентное пространство картинок в CVAE и сравним с картинкой для VAE =)

Опять же, нужно покрасить точки в разные цвета в зависимости от класса.

```
latent_vectors = []
labels_list = []

model.eval()

with torch.no_grad():
    for batch_x, batch_y in test_loader:
        x = batch_x.to(device)
        y = batch_y.to(device)

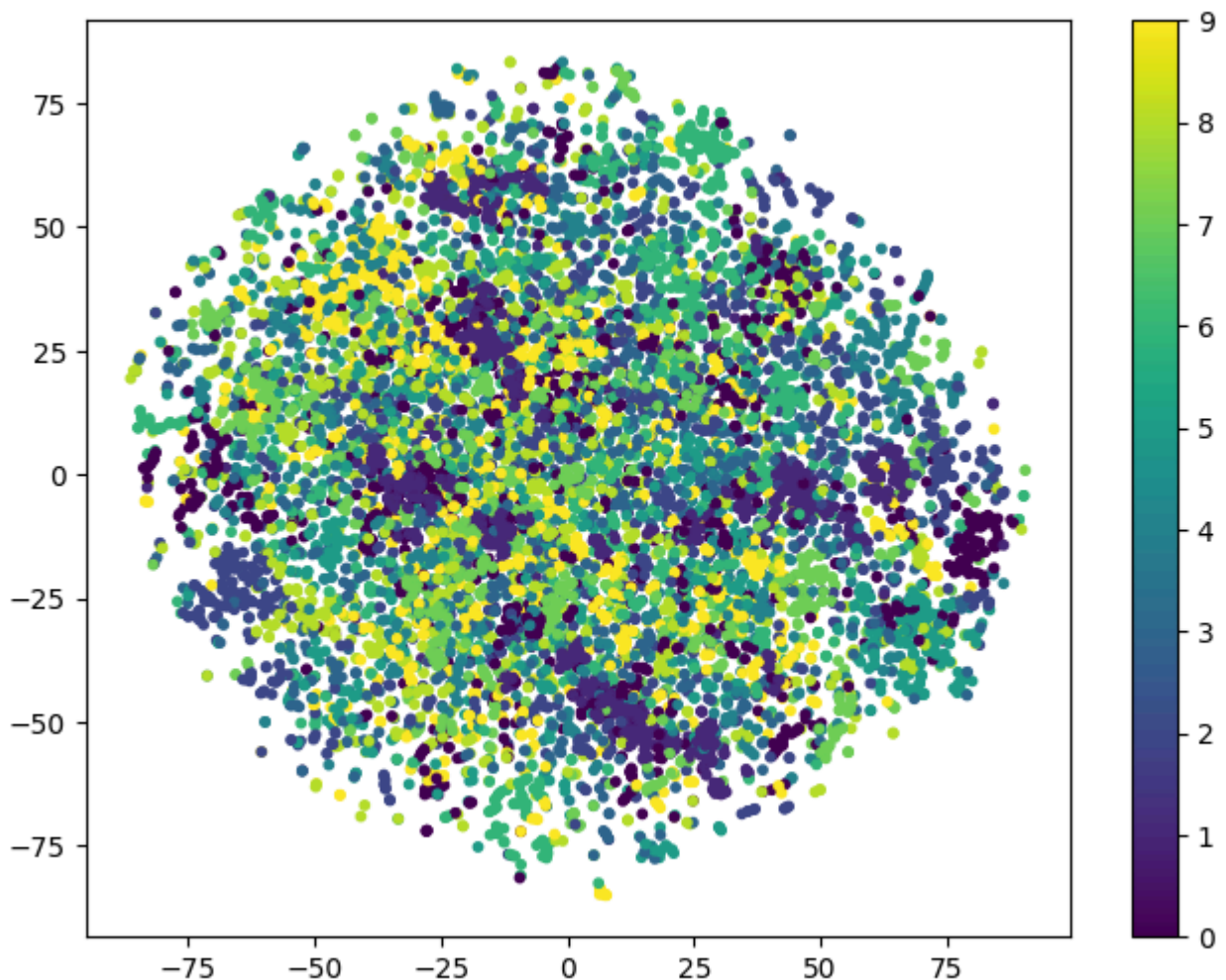
        mu, _, _ = model.encode(x, y)
        latent_vectors.append(mu.cpu().numpy())
        labels_list.append(batch_y.numpy())

latent_vectors = np.concatenate(latent_vectors, axis=0)
labels = np.concatenate(labels_list, axis=0)

tsne = TSNE(n_components=2)
latent_2d = tsne.fit_transform(latent_vectors)
```

```
plt.figure(figsize=(8,6))
scatter = plt.scatter(latent_2d[:, 0], latent_2d[:, 1], c=labels, s=10)
plt.colorbar(scatter)
```


<matplotlib.colorbar.Colorbar at 0x7ae7d27990d0>



Что вы думаете насчет этой картинки? Отличается от картинки для VAE?

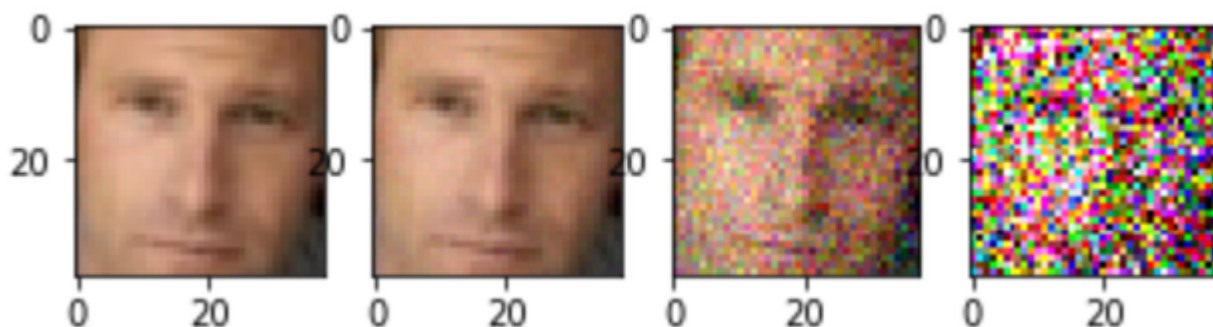
Я переопределял модели, поэтому построить графики рядом не получится, но сразу видно отличия

На графике видно, что латентные представления картинок распределены равномерно, скорее всего это из-за того, что модели не нужно выучивать какой именно это класс, ведь мы передаём его явно

✓ BONUS 1: Denoising (2 балла)

У автоэнкодеров, кроме сжатия и генерации изображений, есть другие практические применения. Про одно из них эта бонусная часть задания.

Автоэнкодеры могут быть использованы для избавления от шума на фотографиях (denoising). Для этого их нужно обучить специальным образом: input картинка будет зашумленной, а выдавать автоэнкодер должен будет картинку без шума. То есть, loss-функция AE останется той же (MSE между реальной картинкой и выданной), а на вход автоэнкодеру будет подаваться зашумленная картинка.



Для этого нужно взять ваш любимый датасет (датасет лиц из первой части этого задания или любой другой) и сделать копию этого датасета с шумом.

В питоне шум можно добавить так:

```
#X_noisy = X + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=X.shape)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test = train_test_split(images, test_size = 0.05, shuffle = True) # 750 на
X_train_tensor = torch.tensor(X_train.transpose(0, 3, 1, 2), dtype=torch.float32) / 255
X_test_tensor   = torch.tensor(X_test.transpose(0, 3, 1, 2), dtype=torch.float32) / 255

from torch.utils.data import TensorDataset, DataLoader
batch_size = 32
train_dataset = TensorDataset(X_train_tensor)
train_dataloader = DataLoader(train_dataset, batch_size = batch_size, shuffle = True )
test_dataset = TensorDataset(X_test_tensor)
test_dataloader = DataLoader(test_dataset, batch_size = batch_size, shuffle = True )
```

```
#просто убрал уменьшение размерности картинки
class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        self.enc = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding = 1),
            nn.ReLU(),
            nn.Conv2d(16, 32, 3, padding = 1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, padding = 1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, padding = 1),
```

```

        nn.ReLU(),
        # output_padding=1
    )

    self.dec = nn.Sequential(
        nn.ConvTranspose2d(128, 128, 3, padding = 1),
        nn.ReLU(),
        nn.ConvTranspose2d(128, 64, 3, padding = 1),
        nn.ReLU(),
        nn.ConvTranspose2d(64, 32, 3, padding = 1),
        nn.ReLU(),
        nn.ConvTranspose2d(32, 16, 3, padding = 1),
        nn.ReLU(),
        nn.ConvTranspose2d(16, 3, 3, padding = 1),
        nn.Sigmoid(),
    )

    def forward(self, x):
        latent_code = self.enc(x)
        reconstruction = self.dec(latent_code)
        # реализуйте forward проход автоэнкодера
        # в качестве возвращаемых переменных -- латентное представление картинки (latent_code)
        # и полученная реконструкция изображения (reconstruction)

        return reconstruction, latent_code

```

```

import torch.optim as optim

criterion = nn.MSELoss()

model = Autoencoder().to(device)

optimizer = optim.Adam(model.parameters(), lr = 3e-4 )

```

```

from tqdm import tqdm
epoch = 25
train_loss = []
test_loss = []
noise_factor = 0.15# со значениями больше, очень тяжело понять что происходит за шумом
val_vis = X_test_tensor[0:1].to(device)
for i in tqdm(range(epoch)):
    model.train()
    epoch_loss = {'train': [], 'test': []}

    for batch in train_dataloader:

        optimizer.zero_grad()
        batch = batch[0].to(device)
        noise = batch + noise_factor * torch.randn_like(batch)

        pred = model(noise)

        loss = criterion(pred[0], batch)

```

```

        loss.backward()
        optimizer.step()
        epoch_loss['train'].append(loss.item())
    eplosstr = np.mean(epoch_loss['train'])
    train_loss.append(eplosstr)

model.eval()
for batch in test_dataloader:
    with torch.no_grad():

        batch = batch[0].to(device)
        noise = batch + noise_factor * torch.randn_like(batch)
        pred = model(noise)
        loss = criterion(pred[0], batch)

        epoch_loss['test'].append(loss.item())
    eplosstest = np.mean(epoch_loss['test'])
    test_loss.append(eplosstest)

val_clean = X_test_tensor[0:1].to(device)
val_noisy = val_clean + noise_factor * torch.randn_like(val_clean)

with torch.no_grad():
    recon_vis, _ = model(val_noisy)

fig, axes = plt.subplots(1, 3, figsize=(6,2))
axes[0].imshow(val_clean[0].cpu().permute(1,2,0).clamp(0,1))
axes[0].set_title('Оригинал')
axes[1].imshow(val_noisy[0].cpu().permute(1,2,0).clamp(0,1))
axes[1].set_title('Зашумлённый')
axes[2].imshow(recon_vis[0].cpu().permute(1,2,0).clamp(0,1))
axes[2].set_title('Восстановленный')
plt.show()

print(f'epoch:{i}, cur train loss: {eplosstr}, curr test loss: {eplosstest}')

```

[Показать скрытые выходные данные](#)

```

fig, axes = plt.subplots(3, 5, figsize=(5 * 2.5, 7))

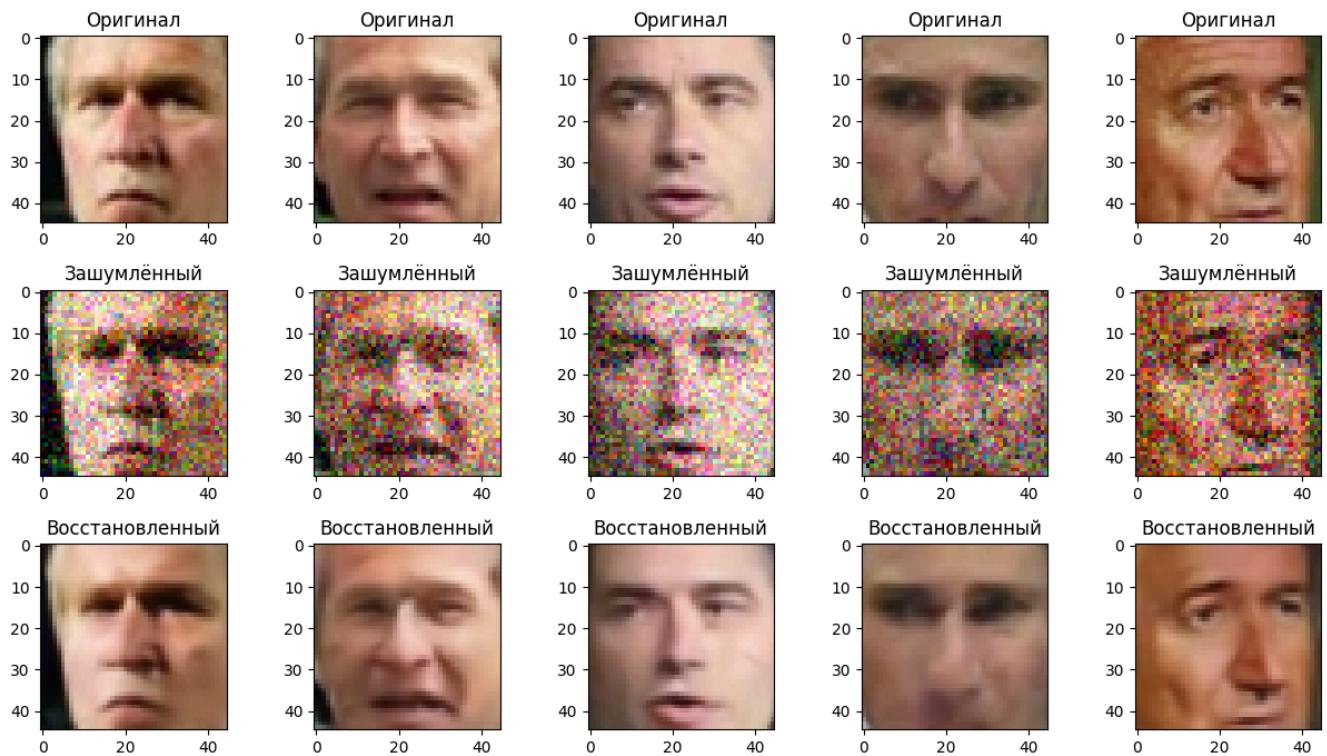
with torch.no_grad():
    for i in range(5):
        clean = X_test_tensor[i:i+1].to(device)
        noisy = clean + noise_factor * torch.randn_like(clean)
        recon, _ = model(noisy)

        axes[0, i].imshow(clean[0].cpu().permute(1, 2, 0))
        axes[0, i].set_title('Оригинал')
        axes[1, i].imshow(noisy[0].cpu().permute(1, 2, 0))
        axes[1, i].set_title('Зашумлённый')
        axes[2, i].imshow(recon[0].cpu().permute(1, 2, 0))
        axes[2, i].set_title('Восстановленный')

plt.tight_layout()
plt.show()

```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers)



✓ BONUS 2: Image Retrieval (2 балла)

Давайте представим, что весь наш тренировочный датасет -- это большая база данных людей. И вот мы получили картинку лица какого-то человека с уличной камеры наблюдения (у нас это картинка из тестового датасета) и хотим понять, что это за человек. Что нам делать? Правильно -- берем наш VAE, кодируем картинку в латентное представление и ищем среди латентных представлений лиц нашей базы самые ближайшие!

План:

1. Получаем латентные представления всех лиц тренировочного датасета
2. Обучаем на них LSHForest (`sklearn.neighbors.LSHForest`), например, с `n_estimators=50`
3. Берем картинку из тестового датасета, с помощью VAE получаем ее латентный вектор
4. Ищем с помощью обученного LSHForest ближайшие из латентных представлений тренировочной базы
5. Находим лица тренировочного датасета, которым соответствуют ближайшие латентные представления, визуализируем!

Немного кода вам в помощь: (feel free to delete everything and write your own)

```
class VAE(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(128 * 6 * 6, 512),
            nn.ReLU()
        )
        self.fc_mu = nn.Linear(512, latent_dim)
        self.fc_logsigma = nn.Linear(512, latent_dim)

        self.decoder_input = nn.Sequential(
            nn.Linear(latent_dim, 512),
            nn.ReLU()
        )
        self.dec = nn.Sequential(
            nn.Linear(512, 128 * 6 * 6),
            nn.ReLU(),
            nn.Unflatten(1, (128, 6, 6)),
            nn.ConvTranspose2d(128, 64, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1, output_padding=0),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 3, stride=2, padding=1, output_padding=0),
            nn.Sigmoid()
        )

    def encode(self, x):
        x = self.enc(x)
        mu = self.fc_mu(x)
        logsigma = self.fc_logsigma(x)
        return mu, logsigma

    def gaussian_sampler(self, mu, logsigma):
        if self.training:
            sigma = torch.exp(0.5 * logsigma)
```

```

        eps = torch.randn_like(sigma)
        return mu + eps * sigma
    else:
        return mu

    def decode(self, z):
        reconstruction = self.decoder_input(z)
        reconstruction = self.dec(reconstruction)
        return reconstruction

    def forward(self, x):
        mu, logsigma = self.encode(x)
        z = self.gaussian_sampler(mu, logsigma)
        reconstruction = self.decode(z)

```

```

criterion = loss_vae

model = VAE().to(device)

optimizer = optim.Adam(model.parameters(), lr = 3e-4 )
from tqdm import tqdm
epoch = 100
train_loss = []
test_loss = []
val_vis = train_dataset[0][0].unsqueeze(0).to(device)
for i in tqdm(range(epoch)):
    model.train()
    epoch_loss = {'train': [], 'test': []}

    for batch in train_dataloader:
        optimizer.zero_grad()
        batch = batch[0].to(device)

        mu, logsigma, recon = model(batch)
        loss = criterion(batch, mu, logsigma, recon)

        loss.backward()
        optimizer.step()
        epoch_loss['train'].append(loss.item())
    eplosstr = np.mean(epoch_loss['train'])
    train_loss.append(eplosstr)

```